

업무 자동화를 위한

바이브 코딩 실습 교육

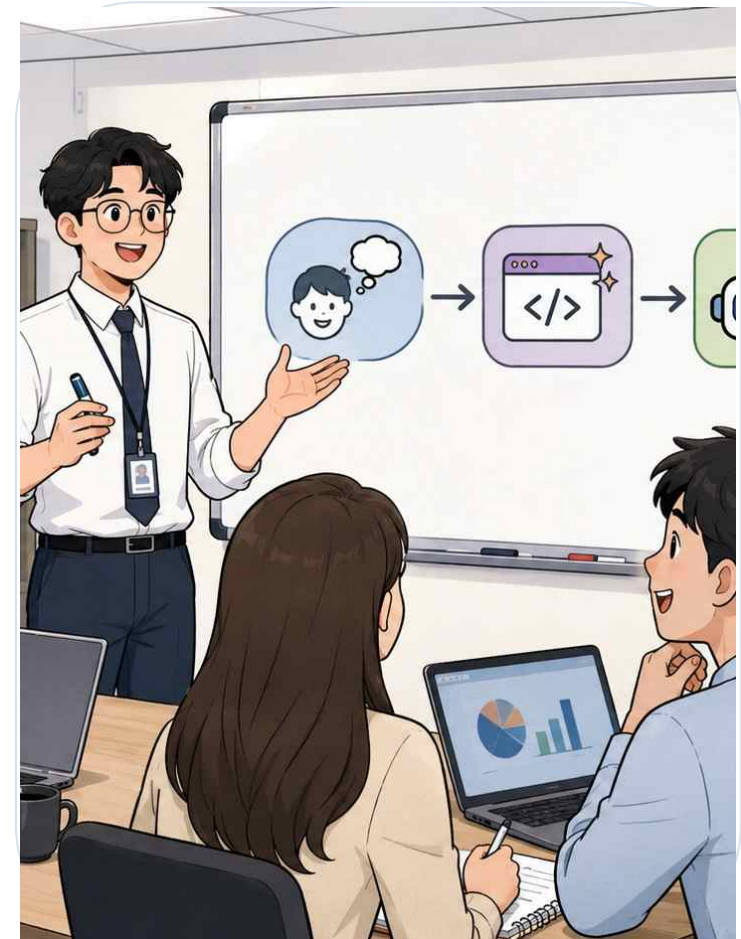
도입부 → 초급 엑셀 취합 → 중급 표도우미

경찰서 250여 개 엑셀 자료 취합 사례로 시작합니다



전체 교육 구성

- 1 도입부: 250개 경찰서 파일 취합의 고통
- 2 초급: 개발환경 세팅과 엑셀 취합 실습
- 3 중급: 표도우미 제작 과정을 단계별로 이해



PART

01

도입부 왜 필요한가?

단순 반복 업무의 고통을 바이브코딩으로 해결



전국 경찰서 자료가 도착했습니다

K-감사관의 폴더에 경찰서 엑셀 파일이 빼곡히 들어 있습니다.

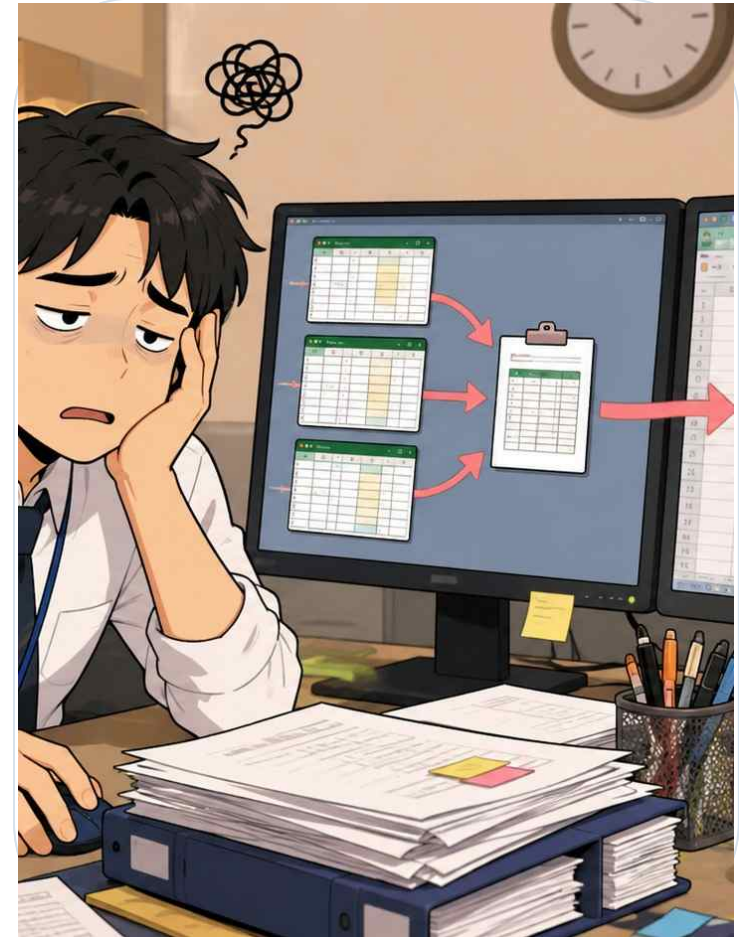
- 전국 경찰서 약 250여 개 엑셀 파일 제출
- 서울중부경찰서.xlsx, 부산중부경찰서.xlsx ...
- 형식은 같지만 파일 수가 많음
- "형식은 같으니까 금방 끝나겠지?"



처음에는 단순히 보였습니다

하나씩 열고 복사하고 붙이면 될 것처럼 보입니다.

- 서울중부경찰서 자료 복사
- 서울종로경찰서 자료 붙여넣기
- 다음은 서울남대문경찰서
- 파일이 10개가 아니라 250개면 이야기가 달라짐



3시간 후, 불안이 시작됩니다

수작업 취합은 시간보다 정확성이 더 큰 문제입니다.

- "부산중부경찰서 파일을 붙였던가?"
- "제목 행이 중간에 한 번 더 들어간 것 같은데..."
- 파일 누락·제목 행 중복·복사 범위 오류 발생 가능
- 한 번 틀리면 검토 시간까지 다시 증가



추가 요구사항이 등장합니다

업무는 한 번에 끝나지 않습니다.

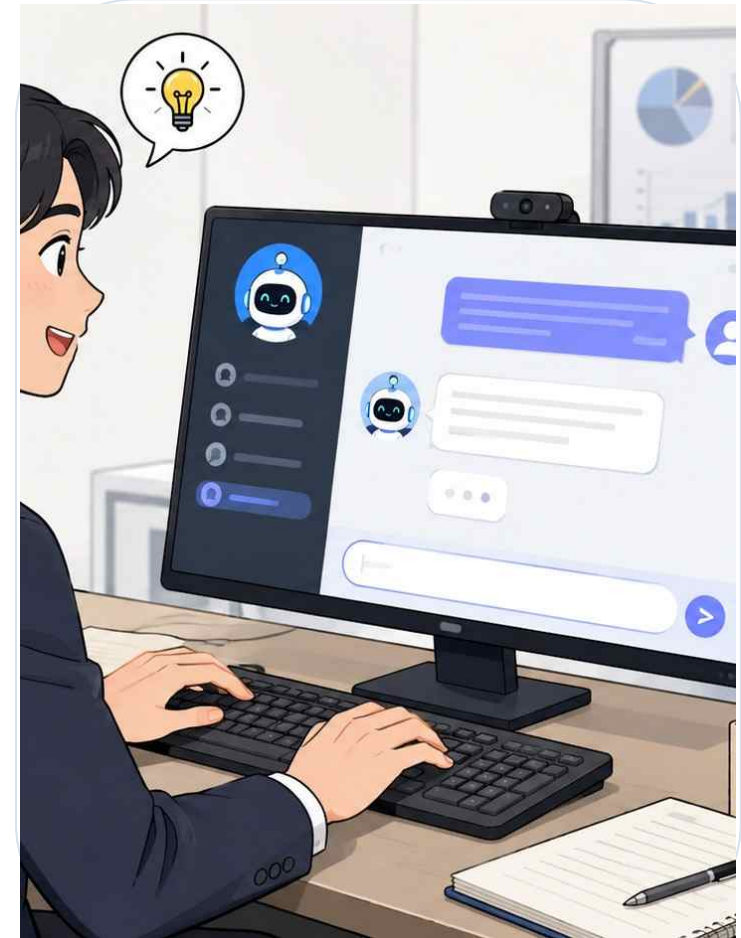
- 수석님: "조치계획 항목도 같이 취합해 주세요."
- 과장님: "원본 파일명도 남겨야 확인하기 좋겠네요."
- 국장님: "지방청별로 구분할 수 있게 지역 열도 추가합니다."
- 요구가 바뀌면 250개를 다시 열어야 함



질문을 바꾸면 자동화가 시작됩니다

바이브 코딩은 이 질문에서 시작됩니다.

- 이 일을 매번 손으로 해야 할까?
- 규칙이 있다면 컴퓨터가 대신할 수 있지 않을까?
- 내가 원하는 절차를 AI에게 설명하면 코드로 만들 수 있지 않을까?
- 반복 작업은 컴퓨터에게, 판단은 사람에게



PART

02

초급 과정 개발환경 세팅

초보자가 가장 많이 막히는 부분을 먼저 해결합니다.

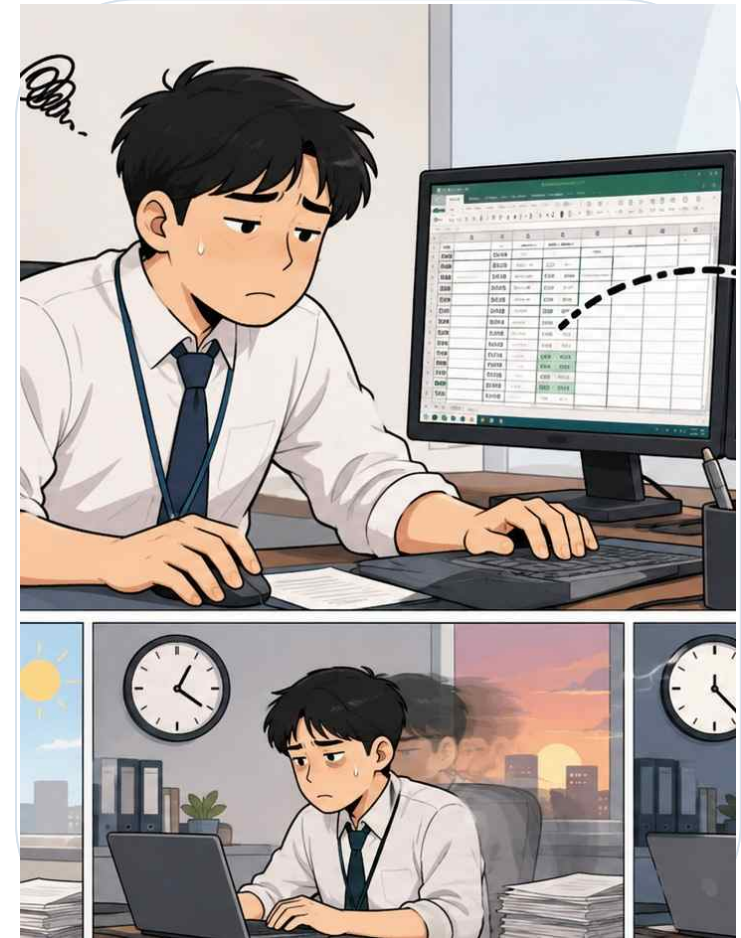
초급 과정



초급은 개발환경 구축이 가장 힘듭니다.

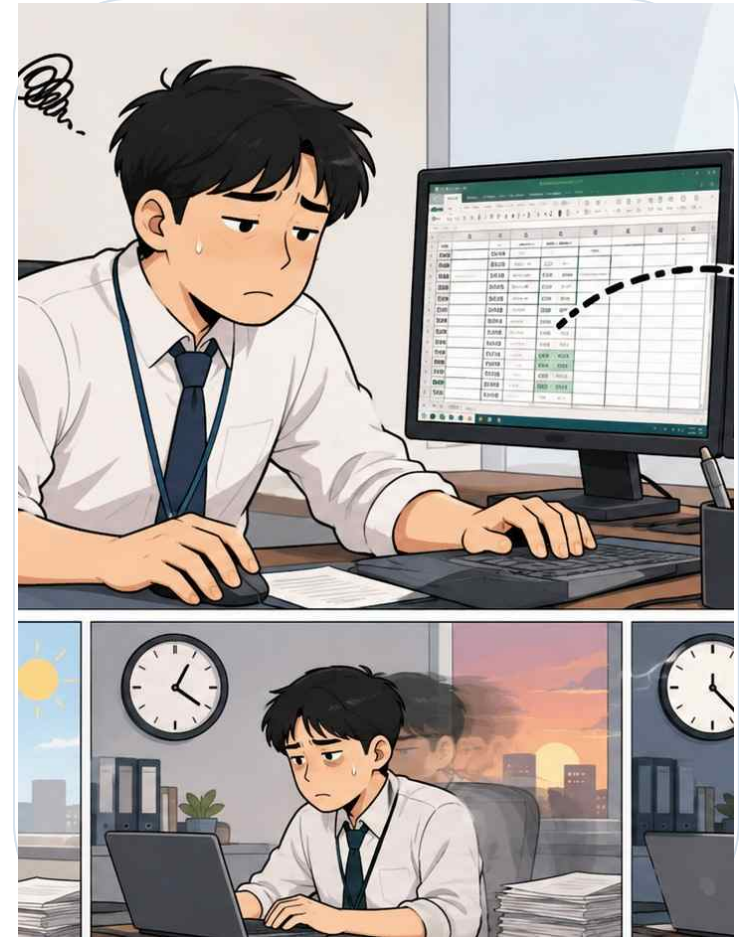
오늘의 목표는 내부망 PC에서 설치부터 코드 실행까지 직접 성공하는 경험입니다.

- Python 설치 확인, VS Code 실행, pip 설치, 코드 실행으로 단순화
- 가상환경은 중급·배포·장기 유지보수 단계에서 권장
- 초보자에게는 가상환경 자체가 첫 장벽이 될 수 있음
- 먼저 성공 경험을 만들고 나중에 환경 관리를 확장



개발환경 세팅 순서

- 1 실습 폴더 만들기
- 2 설치 파일 준비
- 3 Python 설치
- 4 VS Code 설치
- 5 Python 확장 설치
- 6 설치 확인·폴더 열기
- 7 오프라인 라이브러리 설치
- 8 첫 코드 실행·오류 해결

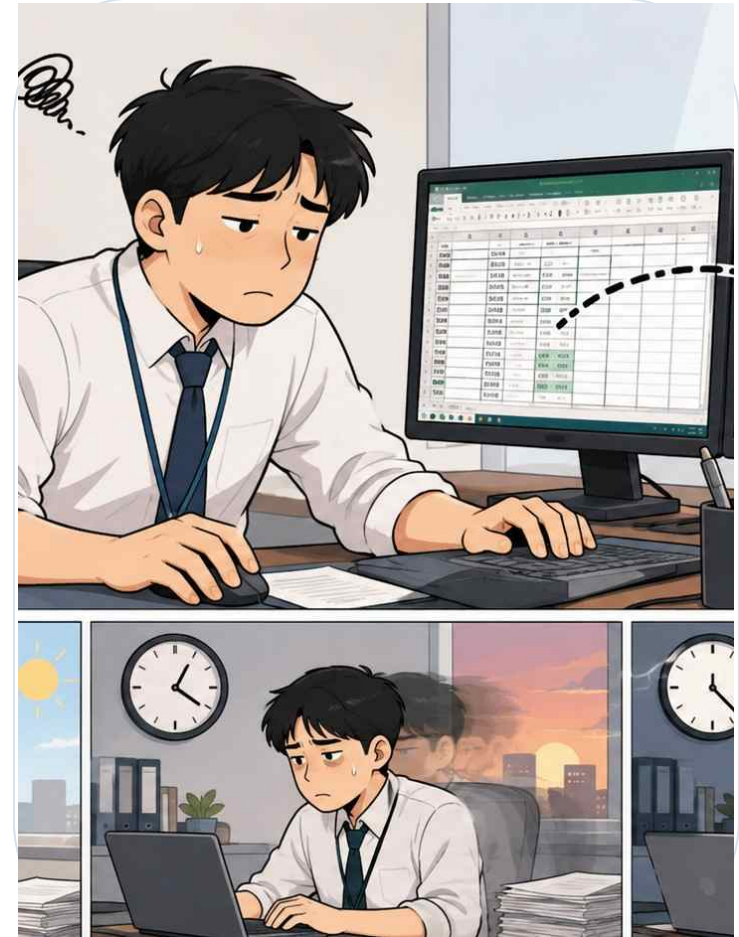


STEP 1

실습 폴더 만들기

- D드라이브에 D:\vibe_excel 폴더 생성
- 그 안에 input, output 폴더 생성
- input에는 경찰서 제출 파일을 넣음
- output은 결과 파일을 분리 보관할 때 사용

```
D:\vibe_excel
├ input
└ output
```

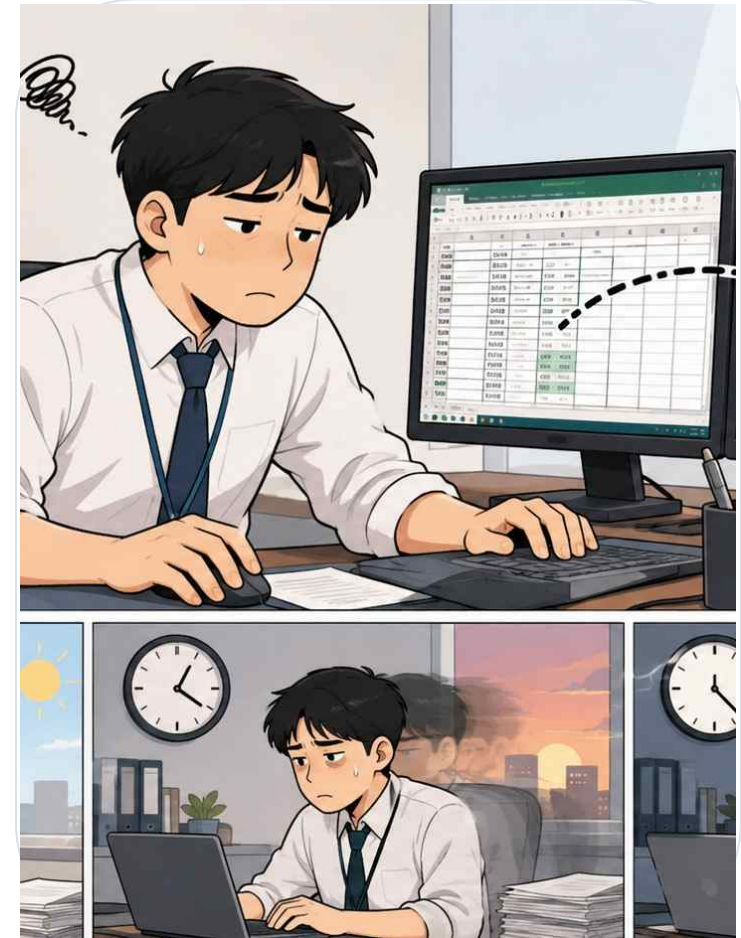


STEP 2

내부망 설치 파일 준비

- 강사가 제공한 설치 폴더를 PC에 복사
- Python 설치 파일과 VS Code 설치 파일 확인
- Python 확장(.vsix)과 패키지 폴더가 있으면 함께 확인
- 파일명은 교육장에서 안내한 버전을 그대로 사용

```
D:\vibe_setup
├─ python-installer.exe
├─ VSCodeSetup.exe
├─ python-extension.vsix
└─ packages
```



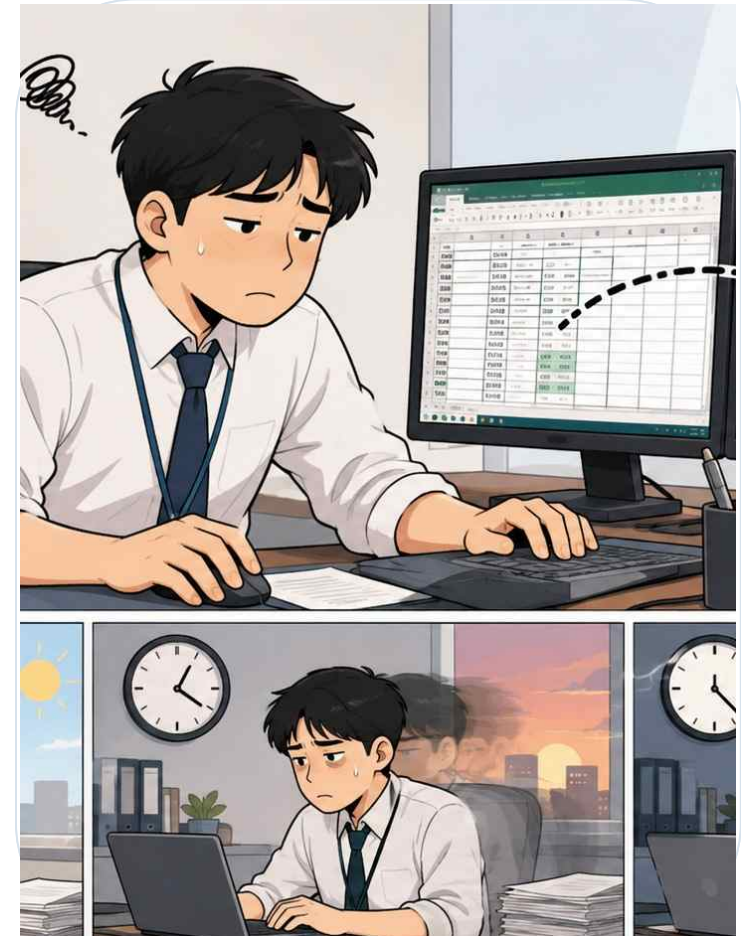
STEP 3

Python 설치하기

- 제공받은 Python 설치 파일을 더블클릭
- Install Now를 눌러 기본 설정으로 설치
- 설치 완료 화면에서 성공 메시지를 확인

가장 중요

☒ Add python.exe to PATH
→ Install Now



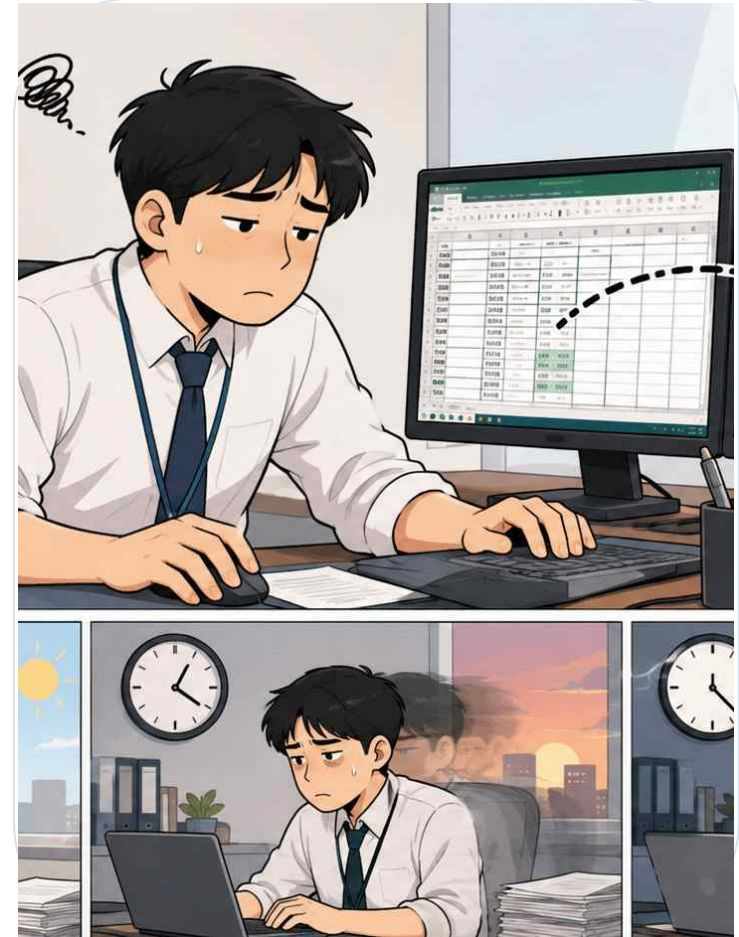
STEP 4

VS Code 설치하기

- 제공받은 VS Code 설치 파일을 더블클릭
- 사용권 계약 동의 후 기본 설치 경로 유지
- 설치 후 Visual Studio Code 실행을 선택

권장 체크

- ☒ PATH에 추가
- ☒ Code로 열기
- ☒ 바탕 화면 바로가기



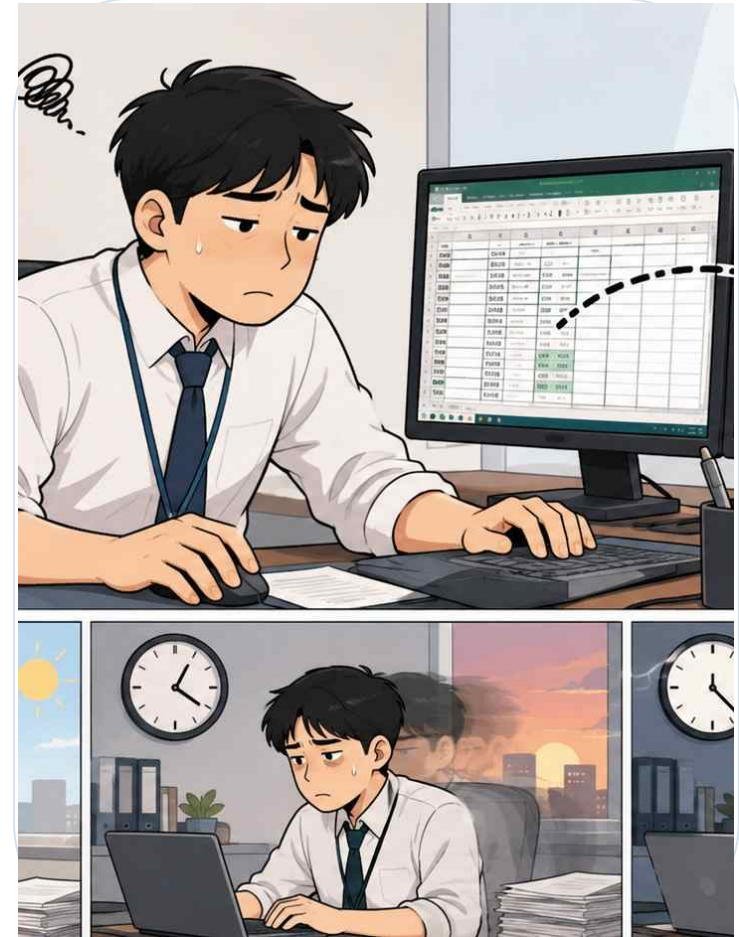
STEP 5

Python 확장 오프라인 설치

- VS Code 왼쪽의 확장 아이콘을 클릭
- 확장 창 오른쪽 위 ... 메뉴를 클릭
- 강사가 제공한 Python 확장 .vsix 파일을 선택

Extensions

- ...
- Install from VSIX...
- python-extension.vsix



STEP 6

Python 설치 확인

명령어	성공 기준
<code>python --version</code>	Python 3.x.x 표시
<code>py --version</code>	Python 3.x.x 표시

명령어가 안 되면 Python 설치 시 "Add python.exe to PATH" 체크 여부를 확인합니다.

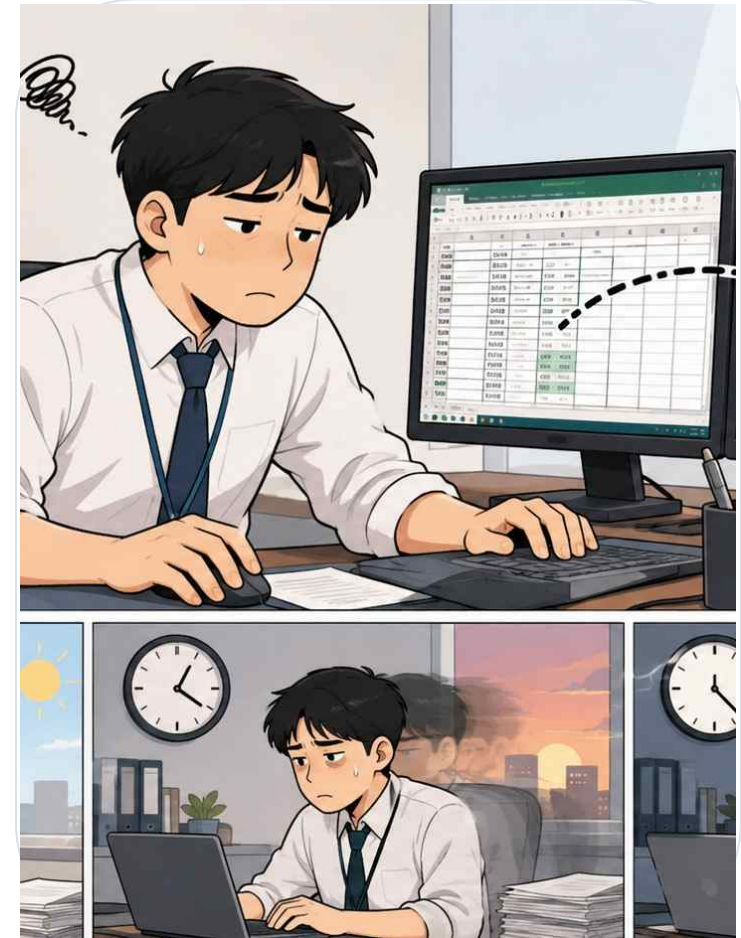


STEP 7

VS Code에서 실습 폴더 열기

- VS Code 실행
- File → Open Folder 선택
- D:\Wvibe_excel 선택
- 왼쪽 탐색기에 input, output 폴더가 보이면 성공

체크 포인트: input, output이 보이지 않으면 폴더를 잘못 연 것입니다.



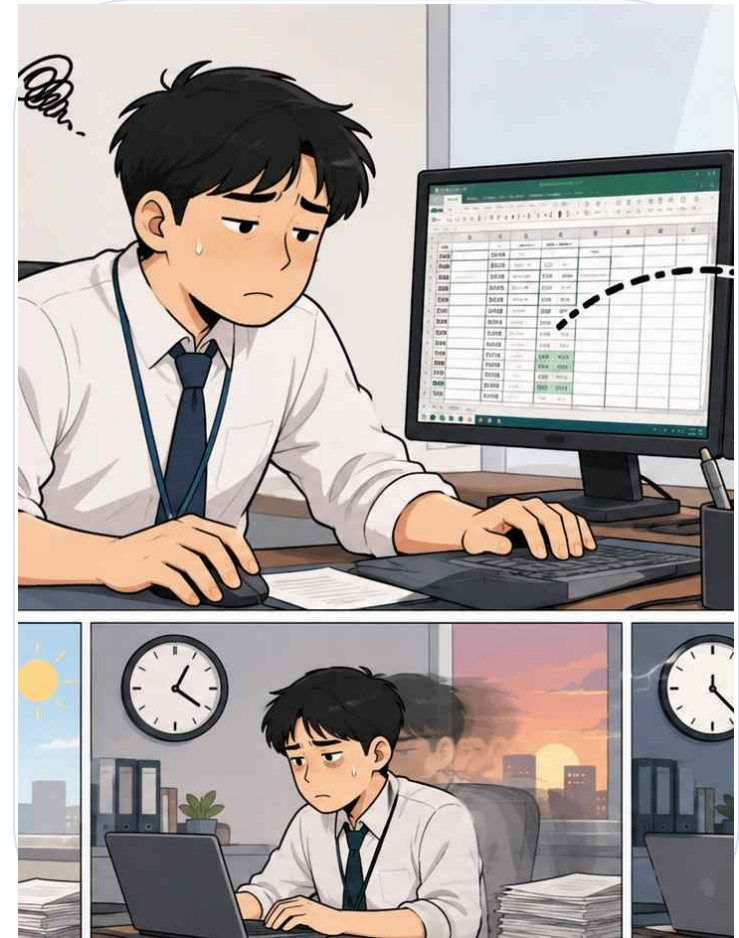
STEP 8

터미널 열기와 위치 확인

- VS Code 상단 메뉴: Terminal → New Terminal
- 터미널 위치가 D:\vibe_excel인지 확인
- 다른 위치라면 `cd /d D:\vibe_excel` 입력

```
D:\vibe_excel>
```

```
cd /d D:\vibe_excel
```



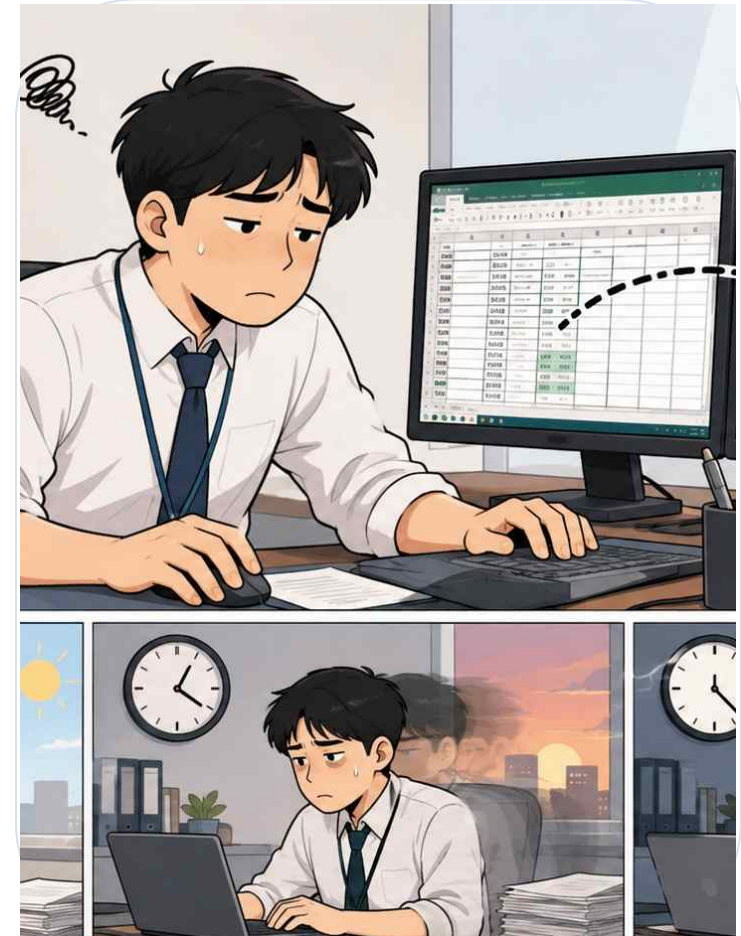
STEP 9

오프라인 라이브러리 설치

인터넷 대신 강사가 제공한 packages 폴더에서 설치합니다.

```
py -m pip install --no-index --find-links=D:\vibe_setup\packages pandas openpyxl  
  
py -m pip list
```

라이브러리	역할
pandas	엑셀 자료를 표처럼 읽고 합치기
openpyxl	xlsx 파일 저장 엔진

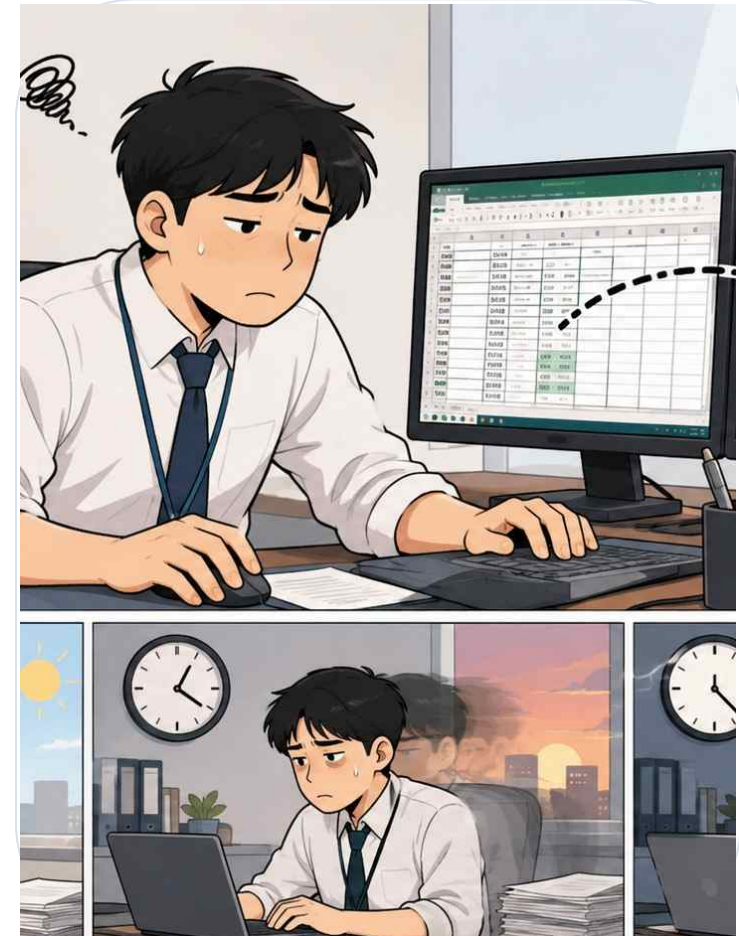


CHECKLIST

개발환경 세팅 체크리스트

- 1 설치 파일 폴더를 PC에 복사했다
- 2 Python 설치 시 PATH를 체크했다
- 3 python 또는 py 버전이 확인된다
- 4 VS Code와 Python 확장을 설치했다
- 5 VS Code에서 D:\vibe_excel을 열었다
- 6 오프라인으로 pandas·openpyxl을 설치했다

여기까지 오면 절반은 성공입니다. 초보자에게는 코드보다 환경 세팅이 더 어렵습니다.

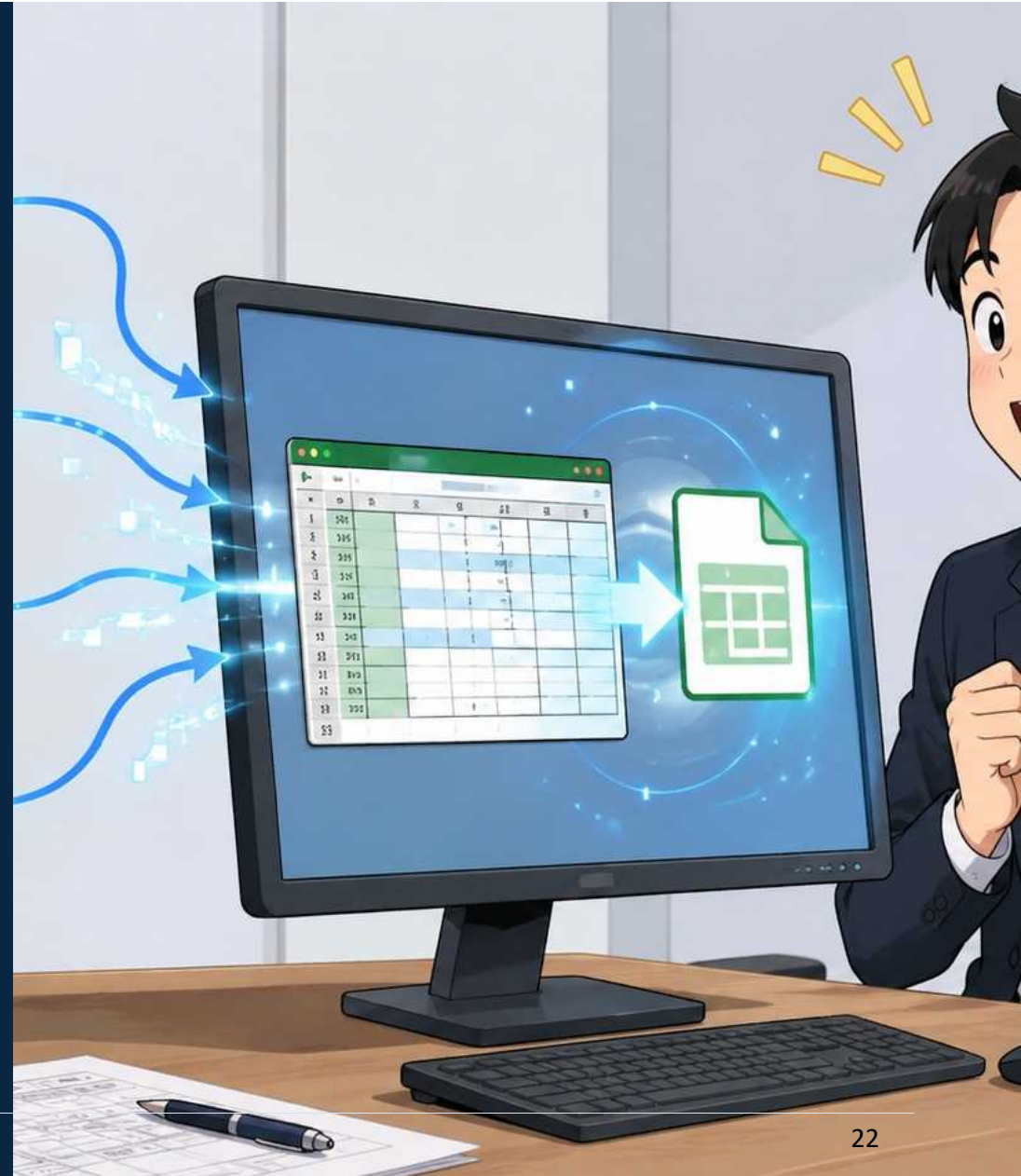


PART

03

초급 과정 엑셀 취합 실습

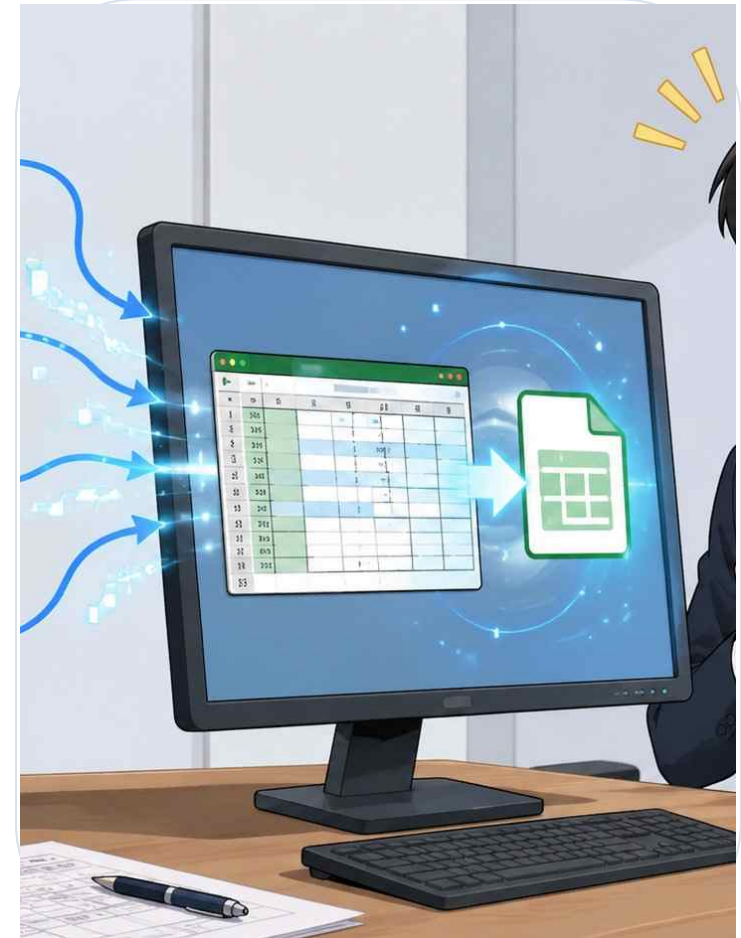
경찰서 엑셀 파일을 하나로 취합하는 프로그램을 직접 만듭니다.



오늘 만들 초급 프로그램

폴더 안의 여러 xlsx 파일을 찾아 하나로 합칩니다.

- 폴더 선택 창을 띄운다
- 선택한 폴더 안의 엑셀 파일을 자동으로 찾는다
- 엑셀 파일을 하나씩 읽는다
- 자료를 하나로 합친다
- 결과 파일을 저장한다

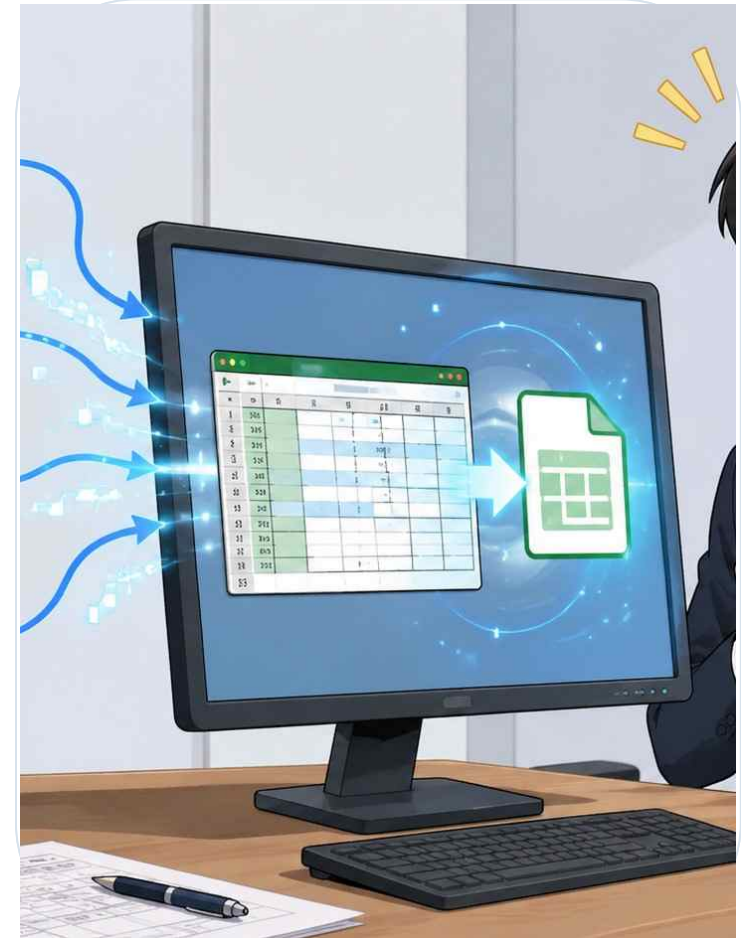


경찰서 제출 파일 예시

실습은 3개 파일로 진행하지만, 같은 코드가 250개 파일에도 적용됩니다.

파일명	기관명	담당자	점검결과	비고
서울중부경찰서.xlsx	서울중부	김하나	적정	제출완료
부산중부경찰서.xlsx	부산중부	이세진	보완	확인필요
대구중부경찰서.xlsx	대구중부	정오름	적정	제출완료

자료가 많아도 형식이 같다면 자동화 대상입니다.

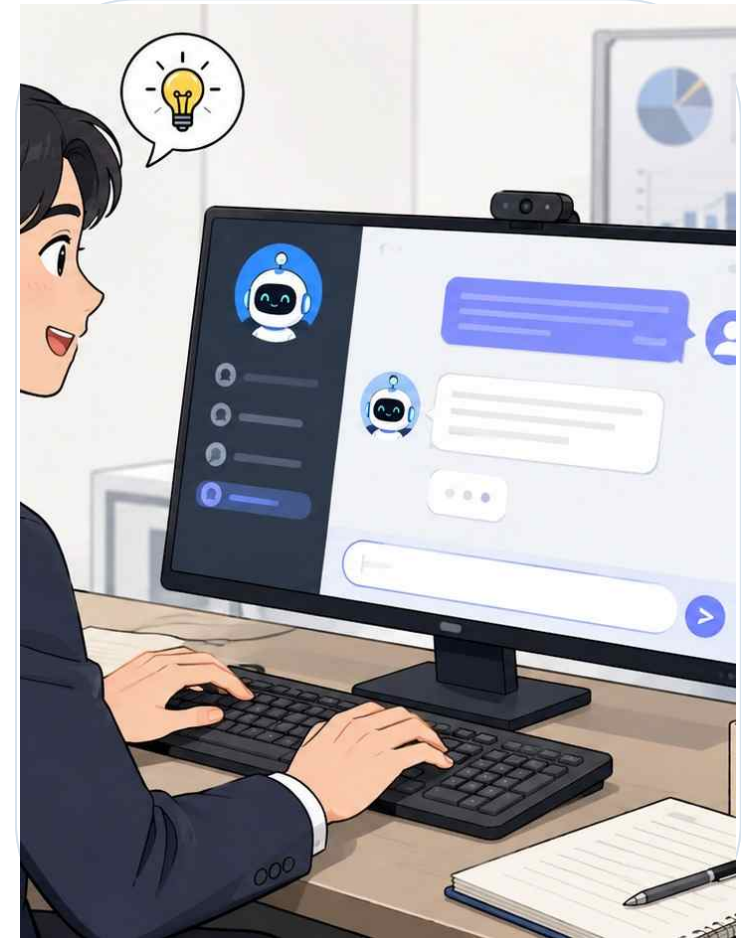


ChatGPT에게 첫 요청하기

형식이 동일한 경찰서 엑셀 파일 여러 개를 하나로 취합하는 파이썬 코드를 만들어줘.

조건

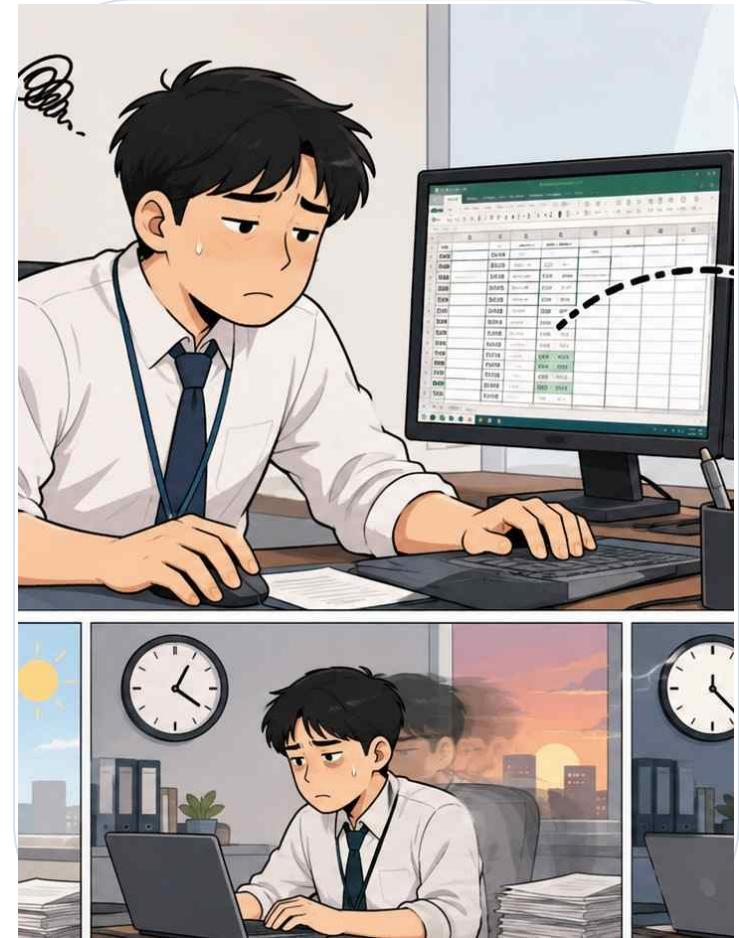
1. Windows에서 실행할 거야.
2. 폴더 선택 창을 띄워줘.
3. 선택한 폴더 안의 `xlsx` 파일을 모두 읽어줘.
4. 엑셀 임시 파일은 제외해줘.
5. 결과 파일은 `merged_result.xlsx`로 저장해줘.
6. 초보자도 이해할 수 있게 주석을 달아줘.



폴더 선택 코드

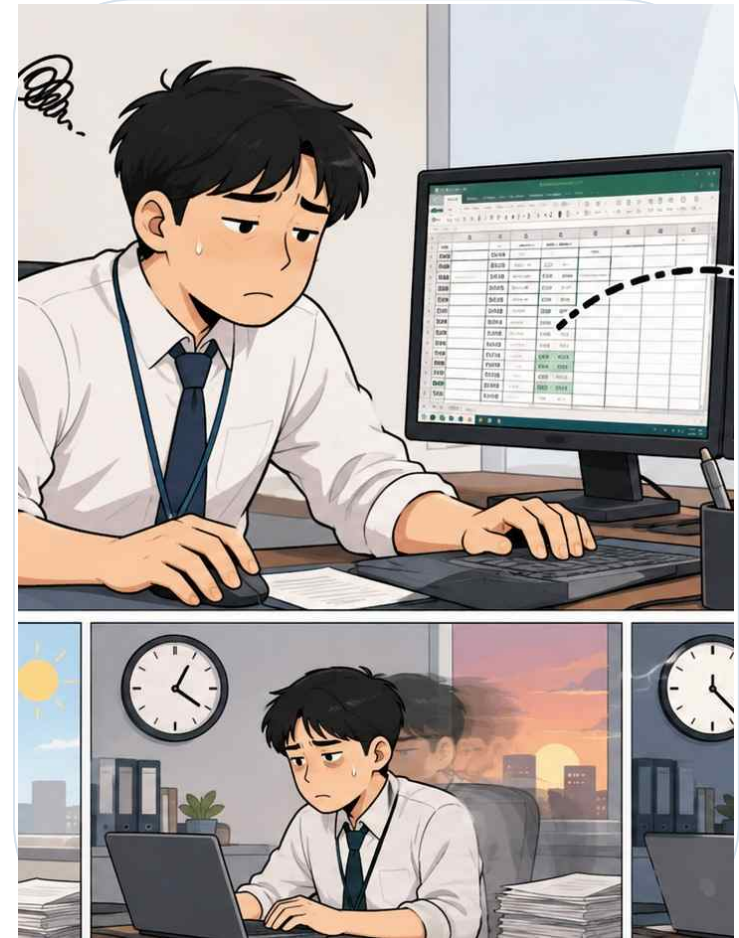
```
import os
import pandas as pd
from tkinter import Tk, filedialog

def select_folder():
    root = Tk()
    root.withdraw()
    folder_path = filedialog.askdirectory(
        title="취합할 엑셀 파일 폴더 선택"
    )
    return folder_path
```



엑셀 파일 찾기와 읽기

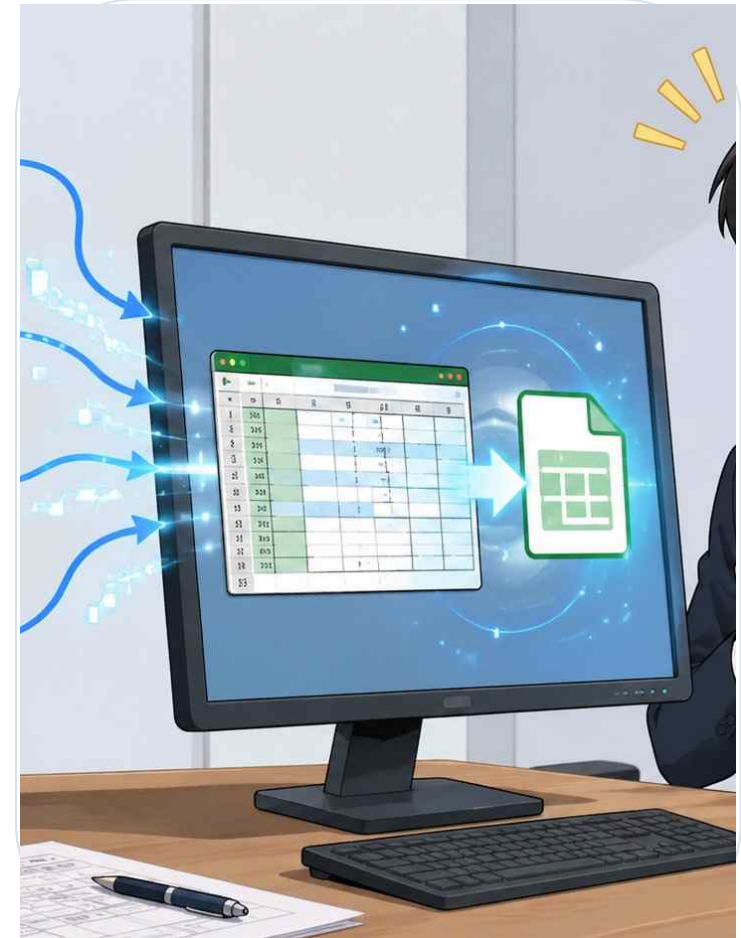
```
def merge_excel_files(folder_path):  
    excel_files = []  
    for file_name in os.listdir(folder_path):  
        if file_name.endswith(".xlsx") and not file_name.startswith("~$"):  
            full_path = os.path.join(folder_path, file_name)  
            excel_files.append(full_path)  
    if not excel_files:  
        print("선택한 폴더에 xlsx 파일이 없습니다.")  
        return  
    excel_files.sort()  
    merged_data = []
```



합치고 저장하기

```
for file_path in excel_files:
    print(f"읽는 중: {os.path.basename(file_path)}")
    data = pd.read_excel(file_path)
    data["원본파일명"] = os.path.basename(file_path)
    merged_data.append(data)

result = pd.concat(merged_data, ignore_index=True)
output_path = os.path.join(folder_path, "merged_result.xlsx")
result.to_excel(output_path, index=False)
print("취합이 완료되었습니다.")
```

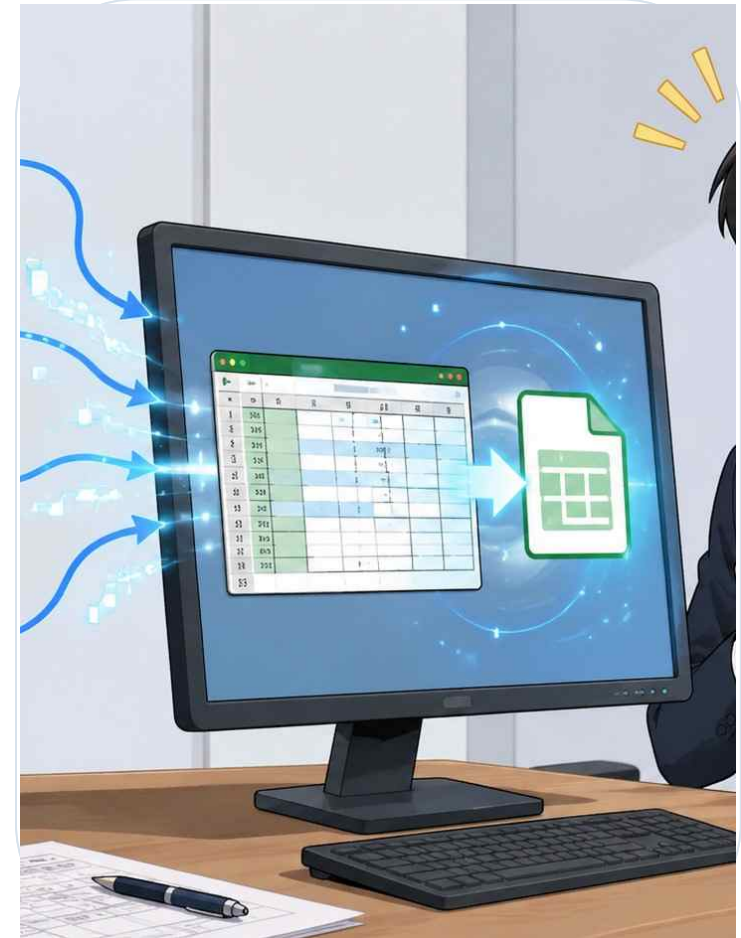


RUN

코드 실행하기

- 터미널에서 `python merge_excel.py` 입력
- 폴더 선택 창에서 `D:\vibe_excel\input` 선택
- input 폴더 안에 `merged_result.xlsx` 생성 확인
- 엑셀을 열어 취합 결과 확인

```
python merge_excel.py
```



RESULT

결과 확인하기

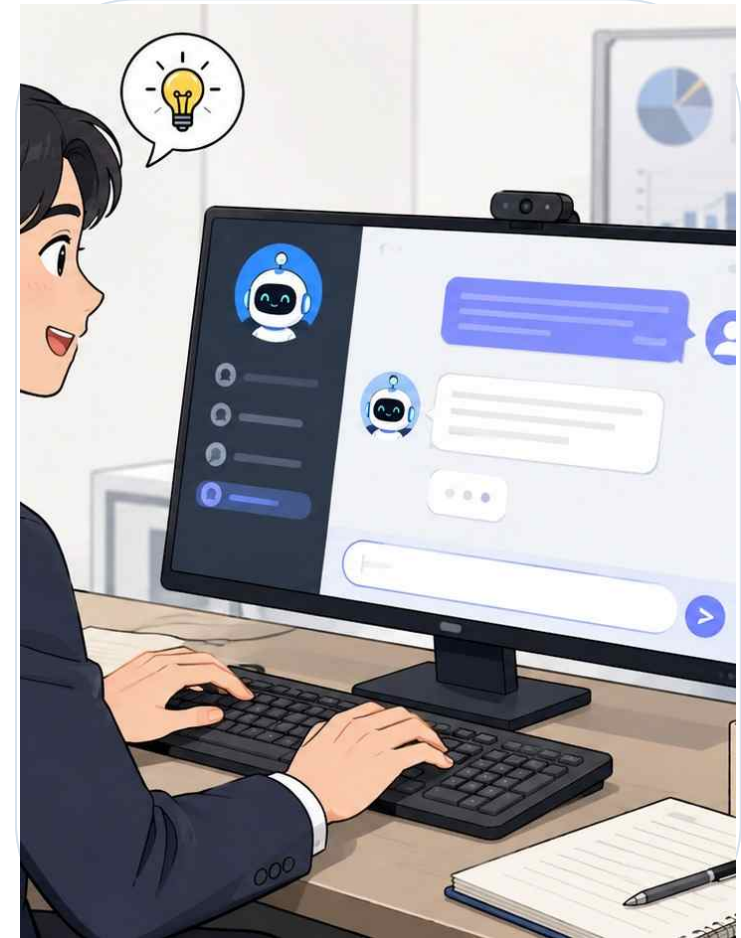
기관명	담당자	점검결과	비고	원본파일명
서울중부	김하나	적정	제출완료	서울중부경찰서.xlsx
부산중부	이세진	보완	확인필요	부산중부경찰서.xlsx
대구중부	정오름	적정	제출완료	대구중부경찰서.xlsx

250개 파일을 하나씩 여는 작업이 버튼 한 번으로 바뀝니다.



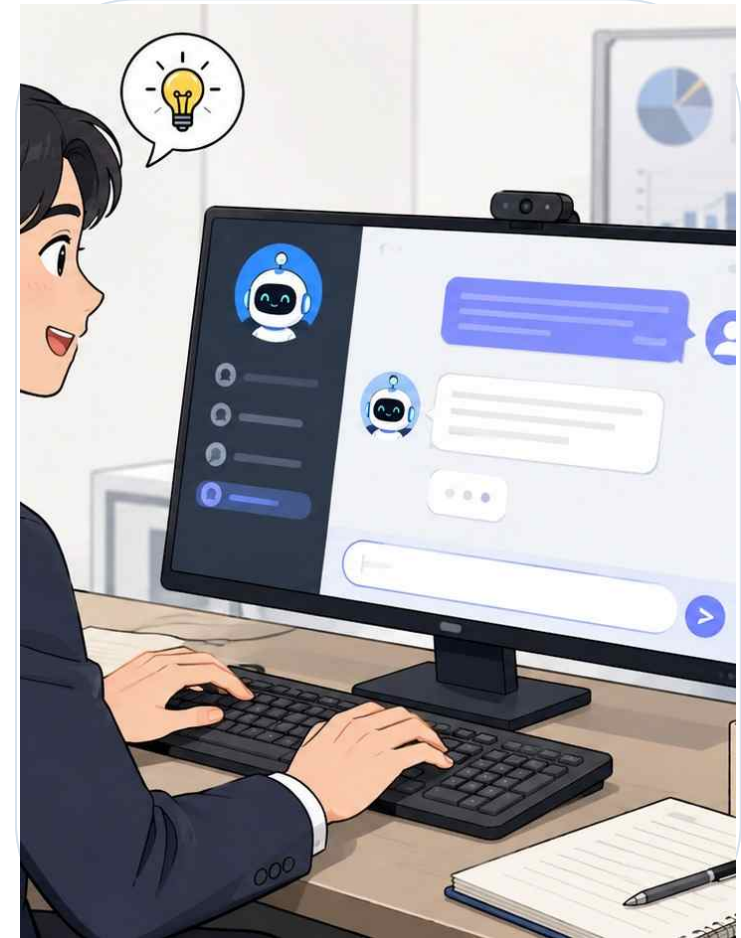
오류 해결: pandas가 없다고 나올 때

- 원인: pandas가 설치되어 있지 않음
- 해결: `pip install pandas openpyxl` 실행
- 설치 후 다시 `python merge_excel.py` 실행



오류 해결: 저장 안 될 때

- 증상: 결과 파일이 저장되지 않거나 권한 오류 발생
- 원인: merged_result.xlsx 파일이 이미 열려 있음
- 해결: 엑셀에서 결과 파일을 닫고 다시 실행
- 파일 이름을 바꾸거나 output 폴더에 저장하도록 수정 가능

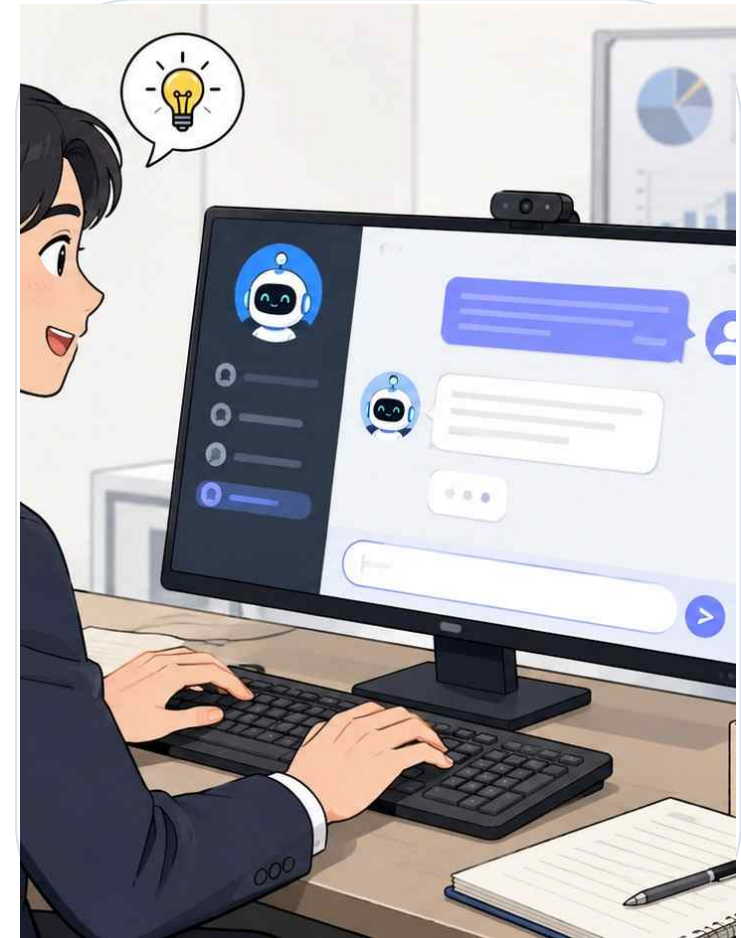


오류 해결: 임시 파일 제외

엑셀 파일을 열어 놓으면 ~\$로 시작하는 임시 파일이 생길 수 있습니다.

```
if file_name.endswith(".xlsx") and not  
file_name.startswith("~$"):  
    full_path = os.path.join(folder_path, file_name)  
    excel_files.append(full_path)
```

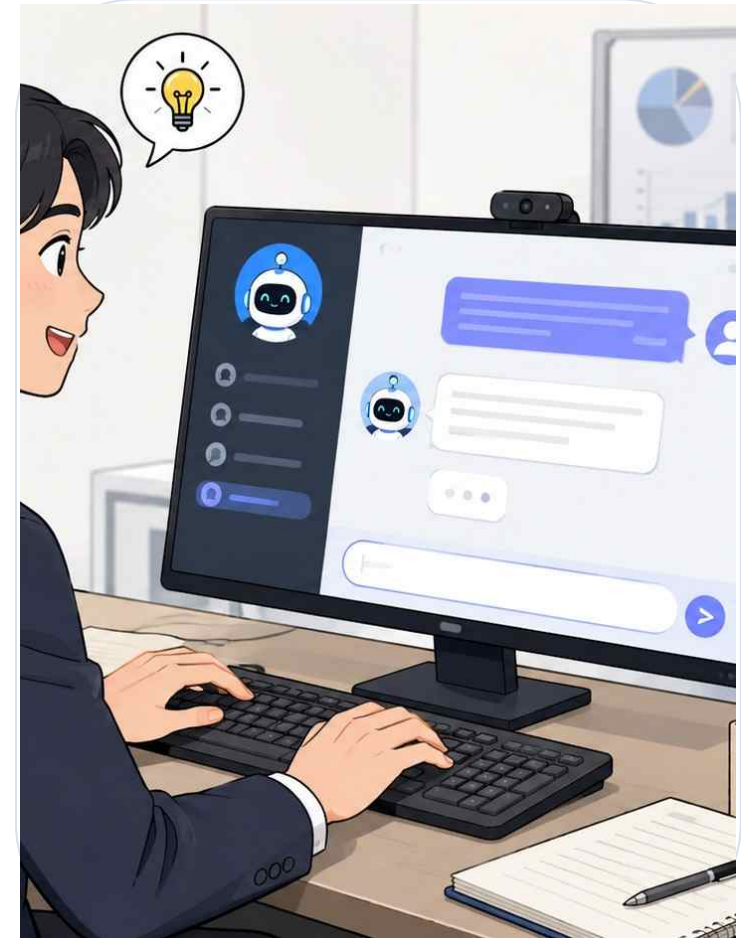
~\$A기관.xlsx는 실제 자료 파일이 아니므로 읽지 않도록 제외합니다.



요구사항 변경: 특정 열만 취합

“기관명, 점검결과, 비고 열만 취합해 주세요.”

```
required_columns = ["기관명", "점검결과", "비고"]  
missing = [c for c in required_columns if c not in data.columns]  
if missing:  
    print("없는 열:", missing)  
    continue  
data = data[required_columns]
```

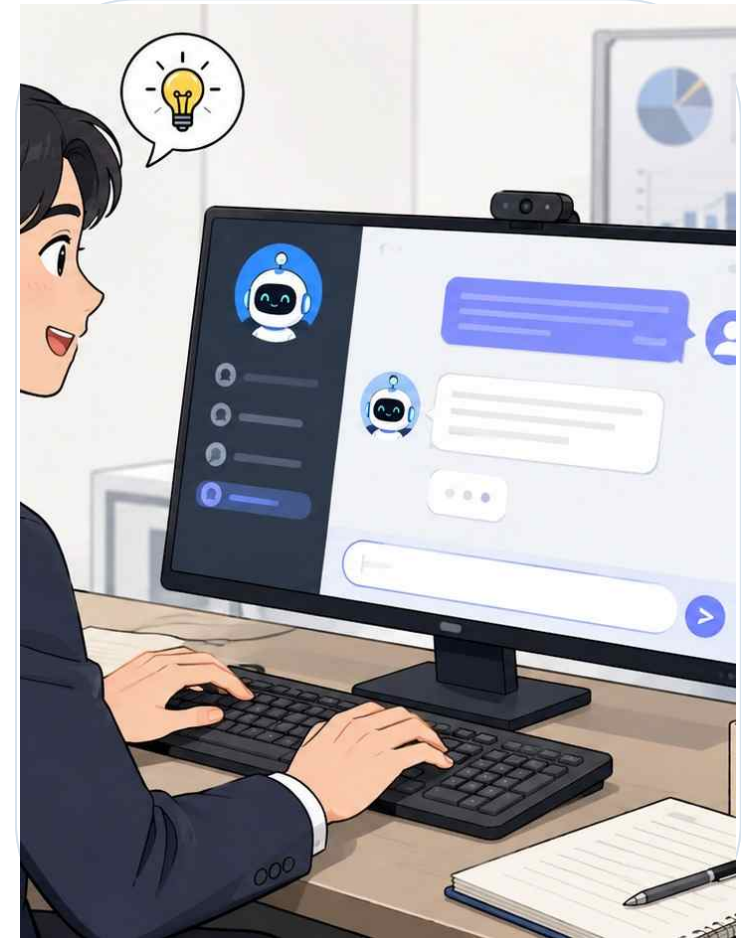


요구사항 변경: 원본파일명 추가

“나중에 어느 파일에서 온 자료인지 알아야 합니다.”

```
data["원본파일명"] = os.path.basename(file_path)
```

원본파일명 열을 넣으면 나중에 오류 자료를 찾거나 기관별 문의를 할 때 유용합니다.



초급 과정 정리

- 1 개발환경을 단순하게 세팅했다
- 2 경찰서 엑셀 파일을 하나로 취합했다
- 3 오류 메시지를 보고 해결했다
- 4 요구사항 변경을 코드에 반영했다
- 5 반복 업무를 컴퓨터에게 맡기는 경험을 했다

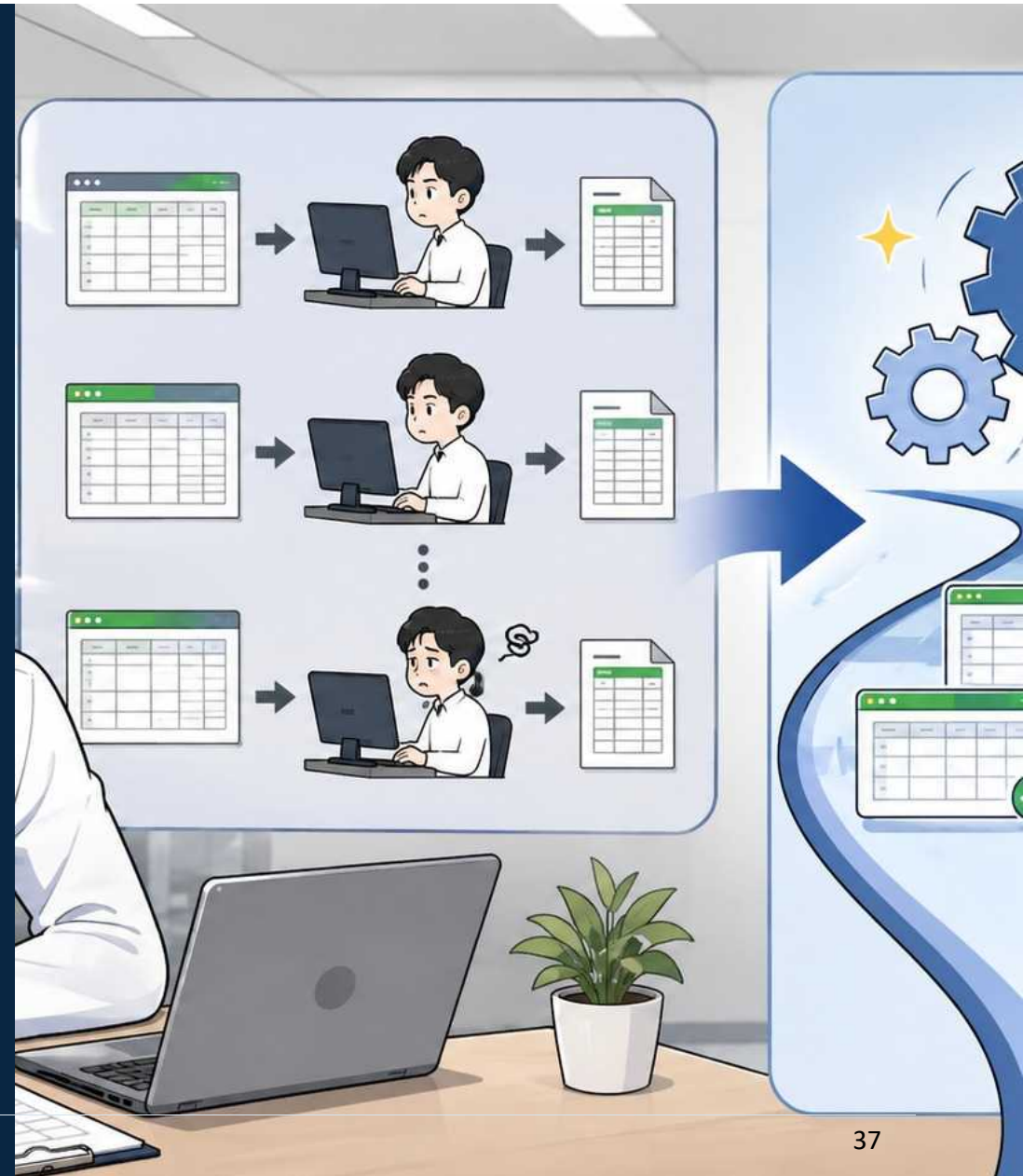


PART

04

중급 과정 표도우미 제작

한글 보고서 표 번호 정리 도구를 단계별로 만들어 갑니다.

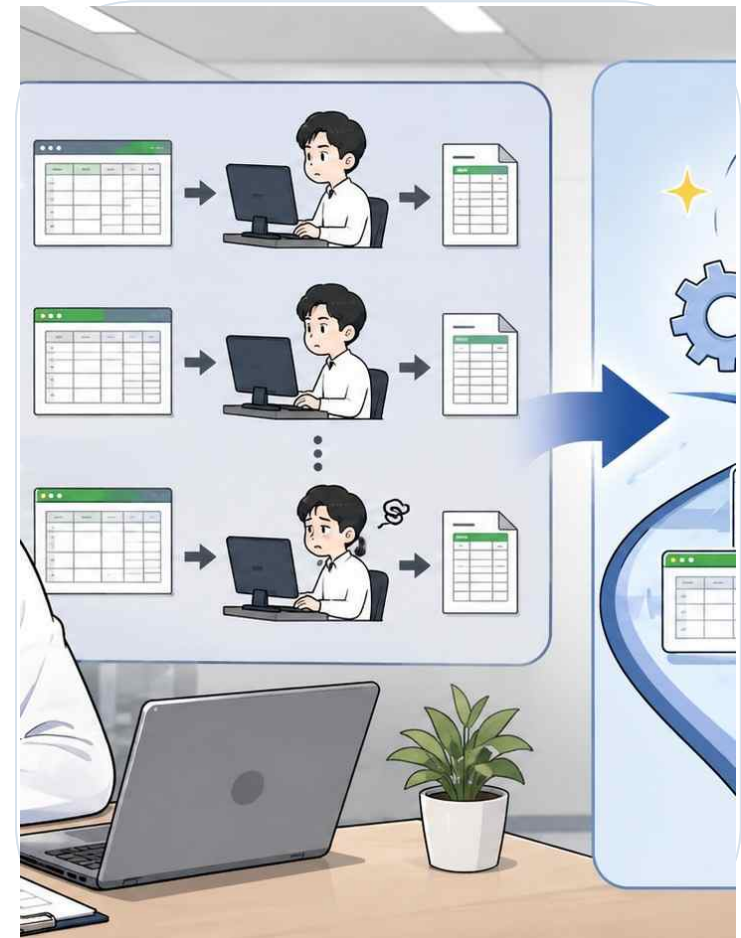


PROBLEM

표도우미는 왜 필요한가?

보고서 중간에 표가 추가되면 뒤 번호를 모두 밀어야 합니다.

- [표 1] 감사 개요
- [표 2] 기관별 제출 현황
- [표 3] 주요 지적사항
- [표 4] 조치계획
- 표가 50개, 100개라면 수작업 수정은 위험

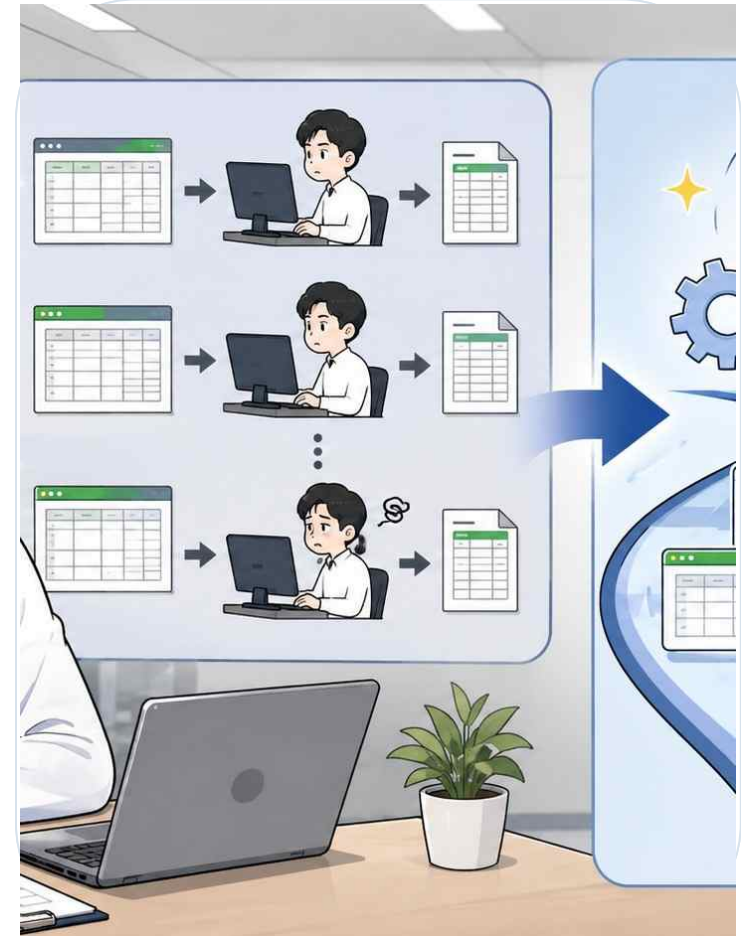


GOAL

표도우미의 최종 목표

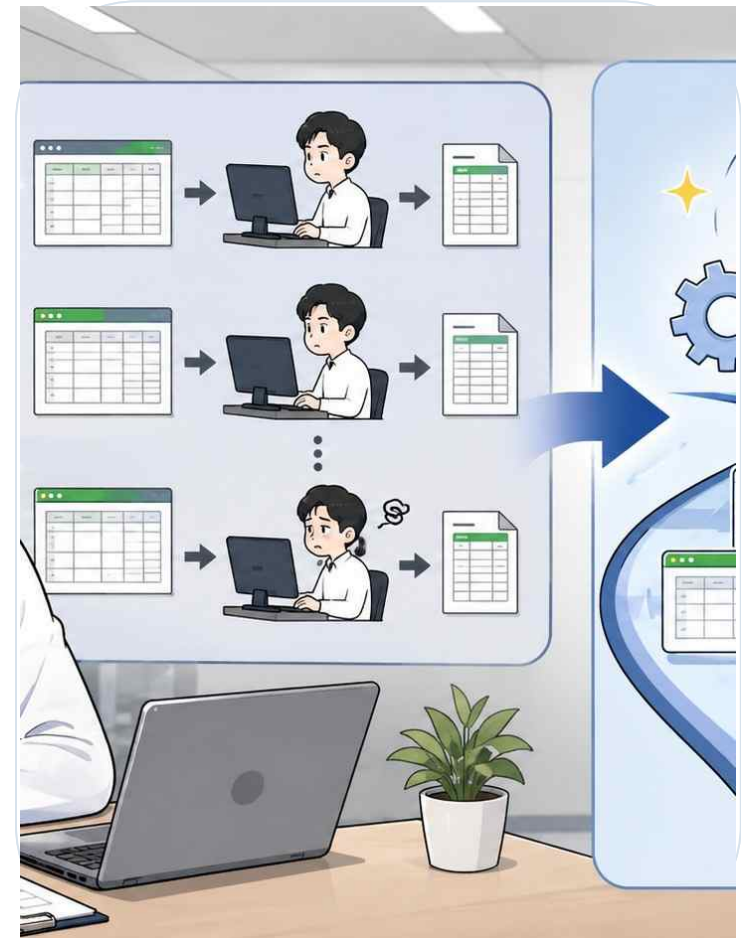
표 번호 관리를 실제 업무 도구로 바꾸는 것이 목표입니다.

- 한글 문서에서 [표 숫자] 제목 형식 찾기
- 찾은 표 제목을 목록으로 표시
- 특정 번호부터 n+ 또는 n- 실행
- [표 0] 새 삽입 표 처리
- 목록 클릭 시 문서 위치 이동
- 나중에는 그림·도표도 처리



중급 과정은 한 번에 완성하지 않습니다

- 1 텍스트 파일에서 표 제목 찾기
- 2 표 번호를 +1, -1 하는 규칙 만들기
- 3 GUI 화면 만들기
- 4 한글 문서에서 텍스트 가져오기
- 5 한글 문서에 수정 결과 반영하기
- 6 예외 처리하기
- 7 표도우미 형태로 정리하기



STEP 1

텍스트에서 표 제목 찾기

처음부터 한글 문서와 연결하지 말고 텍스트 파일로 연습합니다.

sample_report.txt

제1절 배경 및 목적

[표 1] 감사 개요
본문 내용입니다.

[표 2] 기관별 제출 현황
본문 내용입니다.

[표 3] 주요 지적사항

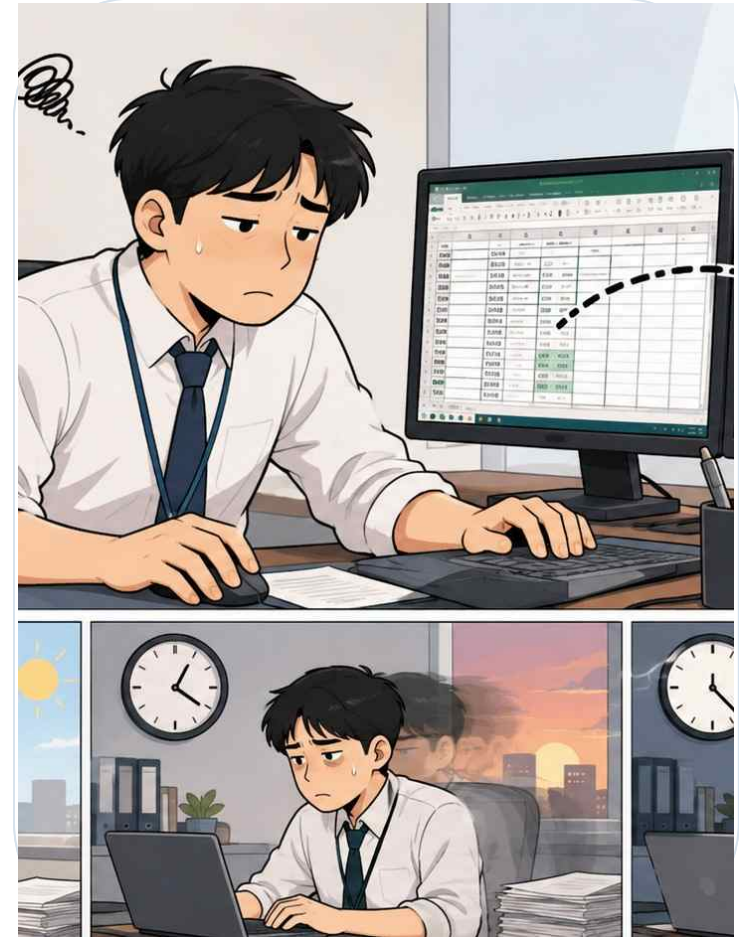
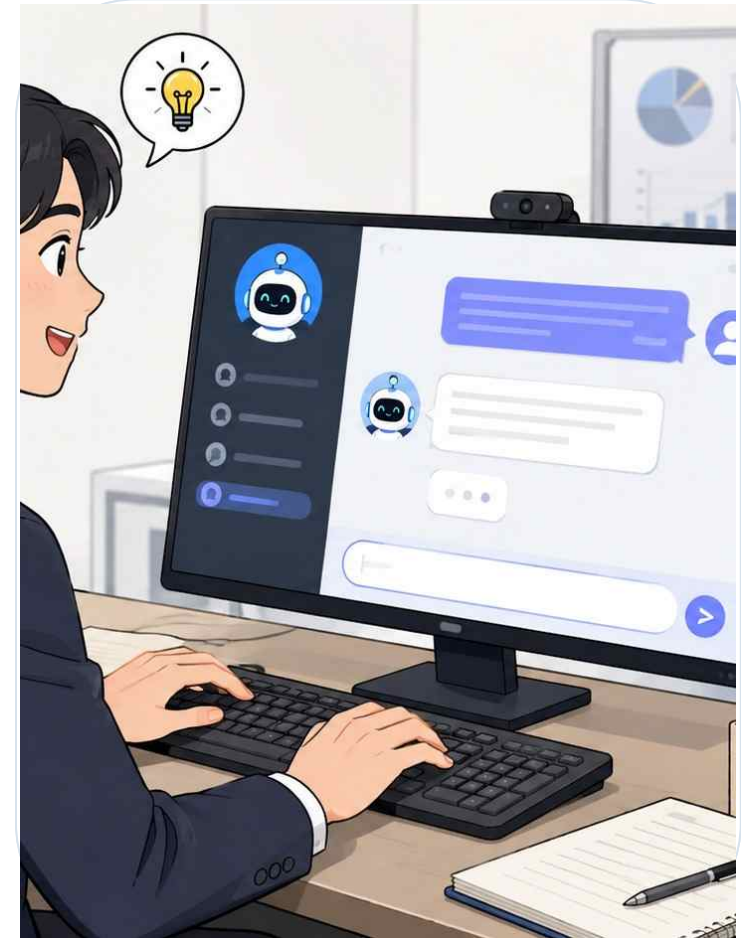


표 제목 찾기 프롬프트

파이썬으로 텍스트 파일에서 [표 숫자] 제목 형식의 줄을 찾아 목록으로 출력하는 코드를 만들어줘.

조건

1. 파일 이름은 sample_report.txt
2. [표 1] 감사 개요 같은 형식 찾기
3. 표 번호와 제목을 분리해서 출력
4. 정규식 사용
5. 초보자도 이해할 수 있게 주석 추가

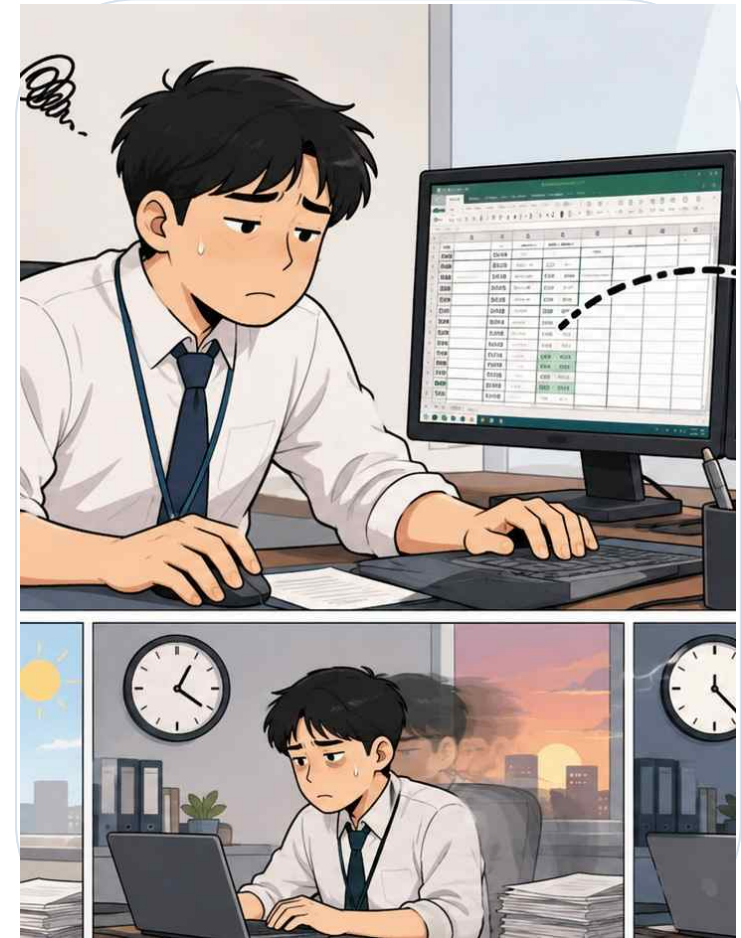


텍스트에서 표 제목 찾는 코드

```
import re
pattern = re.compile(r"^\[표\s*(\d+)\]\s*(.+)$")

with open("sample_report.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()

for line in lines:
    line = line.strip()
    match = pattern.match(line)
    if match:
        table_no = match.group(1)
        title = match.group(2)
        print(f"표 번호: {table_no}, 제목: {title}")
```



STEP 2

특정 번호부터 +1 하기

예: [표 3]부터 뒤 번호를 하나씩 증가시킵니다.

수정 전	수정 후
[표 1] 감사 개요	[표 1] 감사 개요
[표 2] 기관별 제출 현황	[표 2] 기관별 제출 현황
[표 3] 주요 지적사항	[표 4] 주요 지적사항
[표 4] 조치계획	[표 5] 조치계획

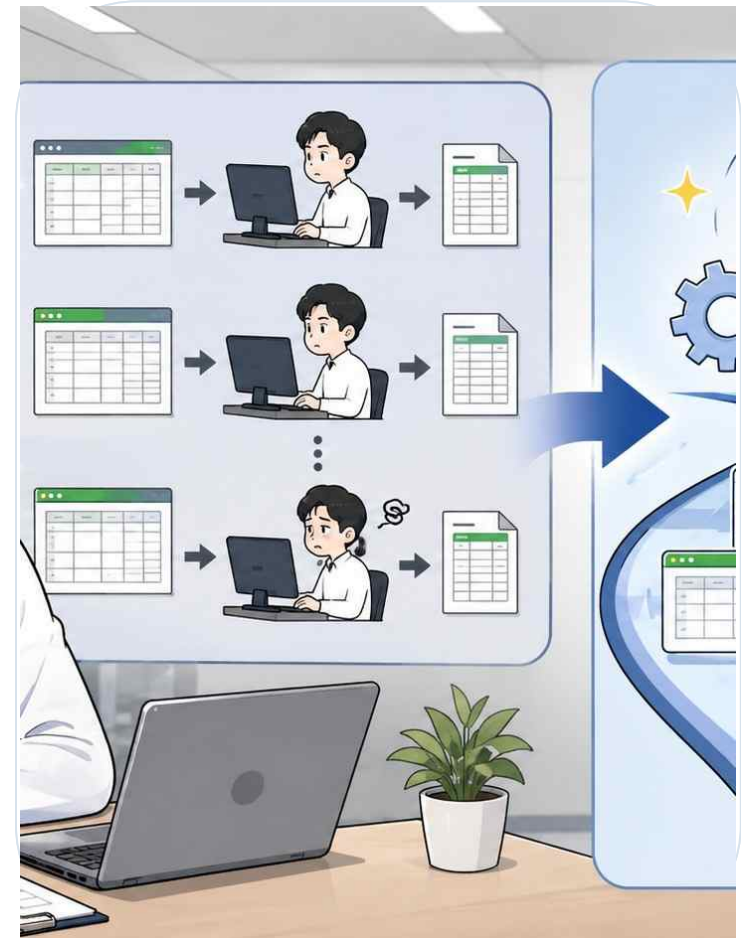
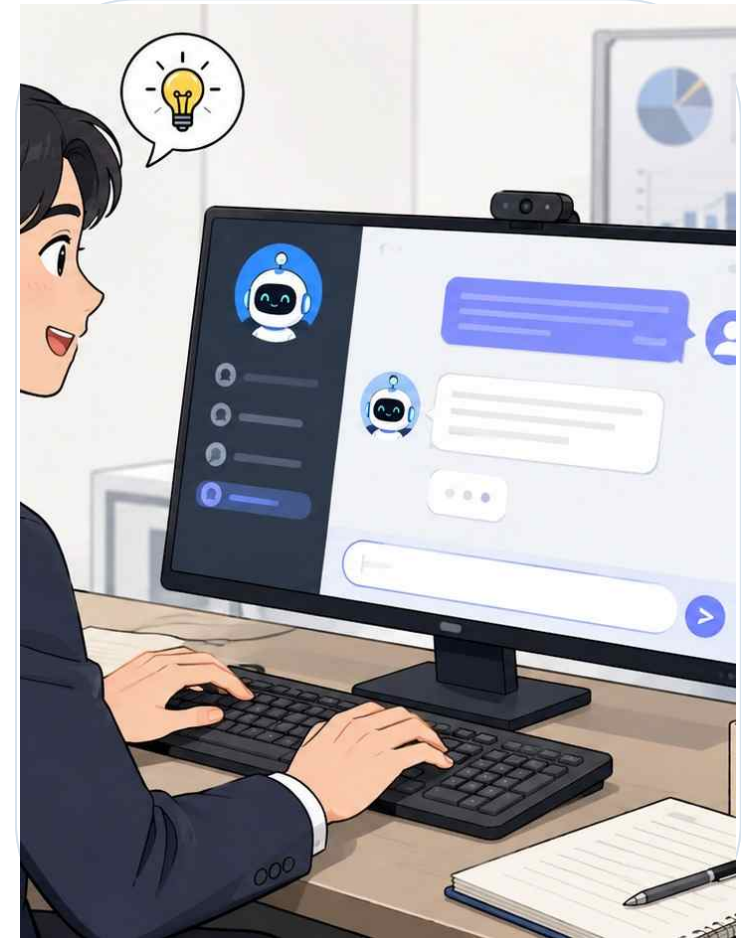


표 번호 +1 프롬프트

sample_report.txt에서 [표 숫자] 제목 형식의 표 번호를 수정하는 파이썬 코드를 만들어줘.

조건

1. 시작 번호를 3으로 정한다.
2. 표 번호가 3 이상이면 +1 한다.
3. 수정 결과는 sample_report_updated.txt로 저장한다.
4. 초보자도 이해할 수 있게 주석을 달아준다.

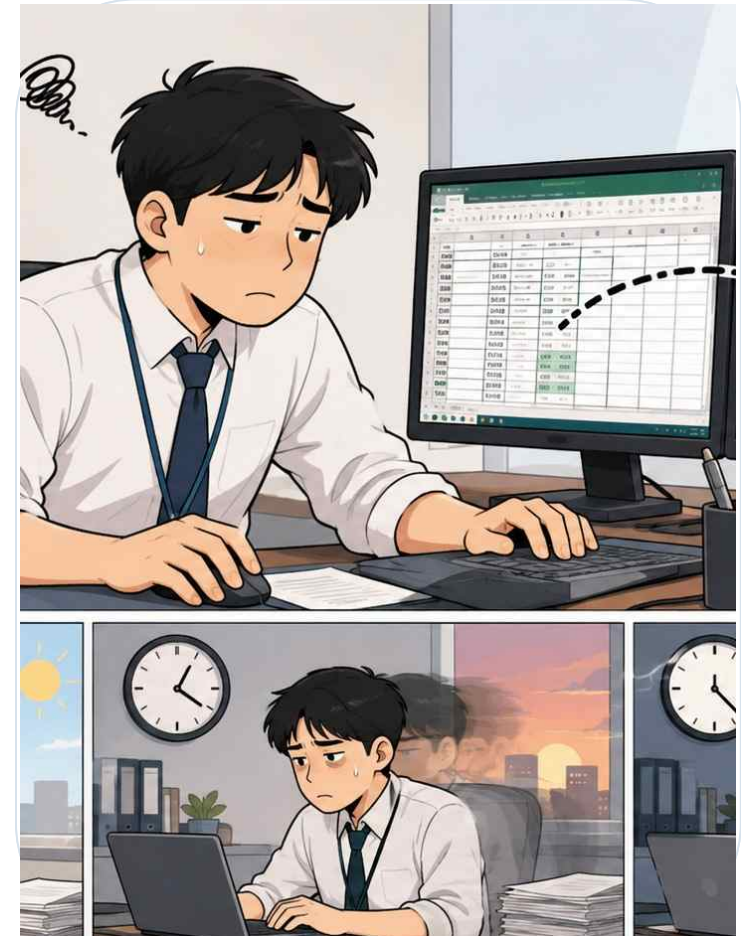


CODE

표 번호 +1 코드

```
start_no = 3
shift = 1

for line in lines:
    stripped = line.strip()
    match = pattern.match(stripped)
    if match:
        table_no = int(match.group(1))
        title = match.group(2)
        if table_no >= start_no:
            table_no = table_no + shift
            updated_lines.append(f"[표 {table_no}] {title}\n")
        else:
            updated_lines.append(line)
```

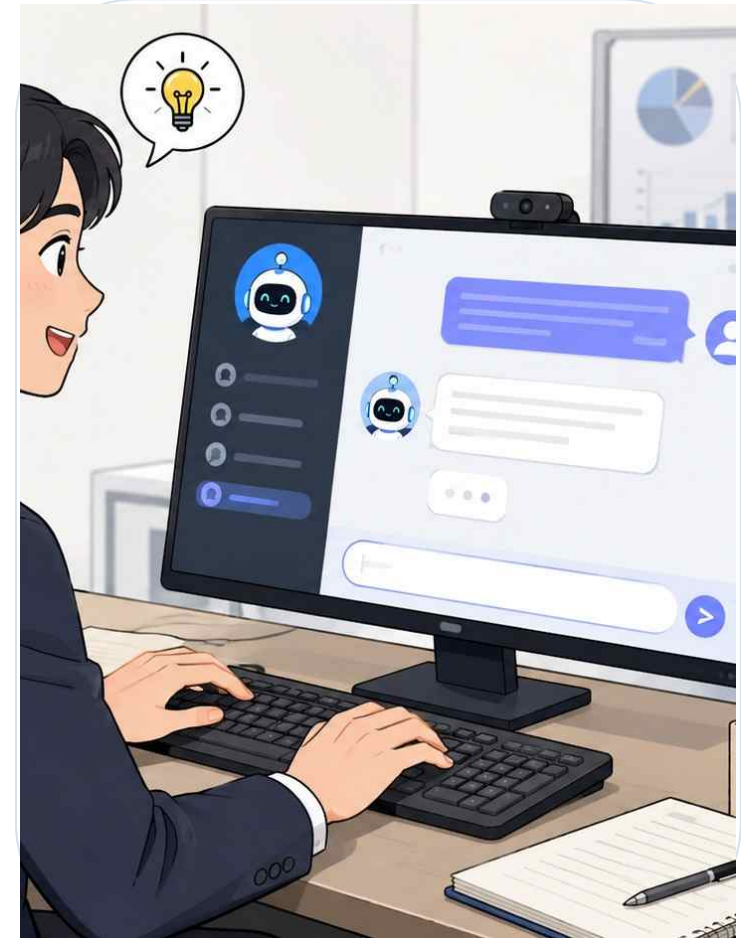


STEP 3

GUI 만들기

매번 코드에서 `start_no`를 고치지 않도록 화면을 만듭니다.

- 시작 번호 입력칸
- n+ 버튼
- n- 버튼
- 실행 결과 메시지
- tkinter로 간단한 업무용 창 제작

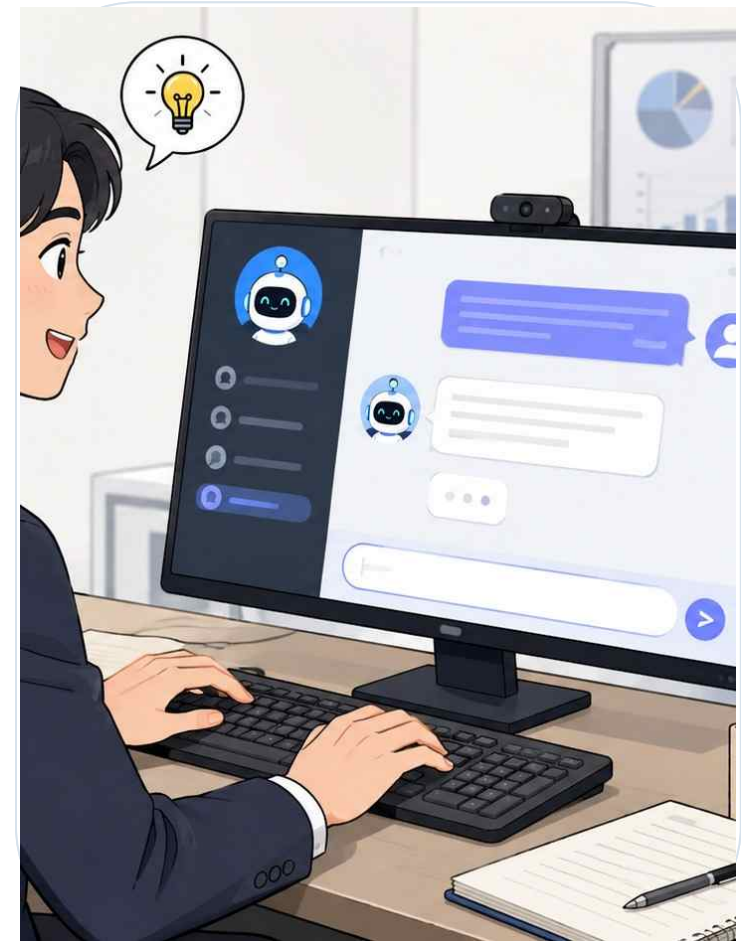


GUI 제작 프롬프트

방금 만든 표 번호 수정 코드를 tkinter GUI로 바꿔줘.

조건

1. 시작 번호를 입력할 수 있게 해줘.
2. n+ 버튼을 누르면 해당 번호 이상을 +1 해줘.
3. n- 버튼을 누르면 해당 번호 이상을 -1 해줘.
4. 결과 파일은 sample_report_updated.txt로 저장해줘.
5. 초보자도 이해할 수 있게 주석을 달아줘.

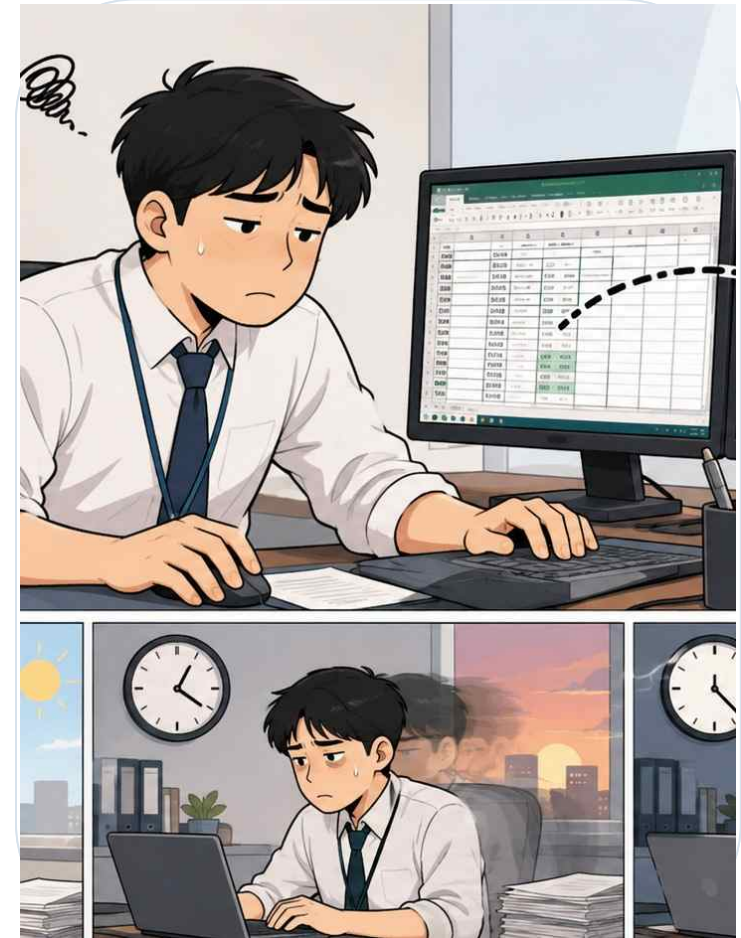


GUI 기본 코드 구조

```
import tkinter as tk
from tkinter import messagebox

def update_table_numbers(shift):
    start_no = int(entry_start_no.get())
    messagebox.showinfo("완료", "수정 완료")

root = tk.Tk()
root.title("표도우미 연습")
entry_start_no = tk.Entry(root)
entry_start_no.pack()
tk.Button(root, text="n+", command=lambda: update_table_numbers(1)).pack()
tk.Button(root, text="n-", command=lambda: update_table_numbers(-1)).pack()
root.mainloop()
```

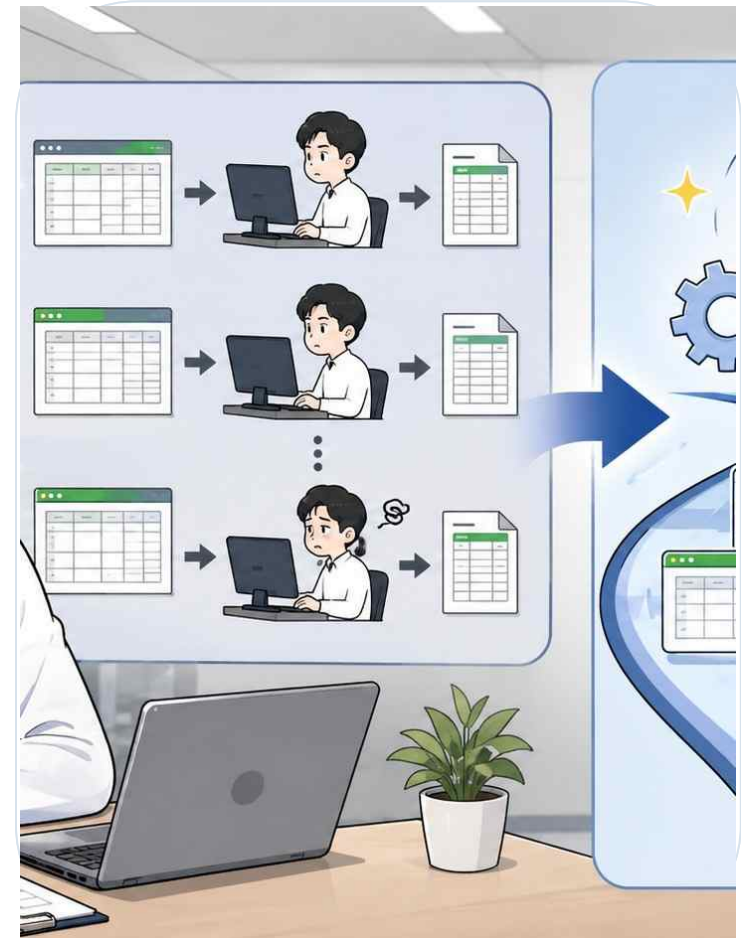


STEP 4

한글 문서와 연결하기

여기서부터는 난이도가 올라갑니다.

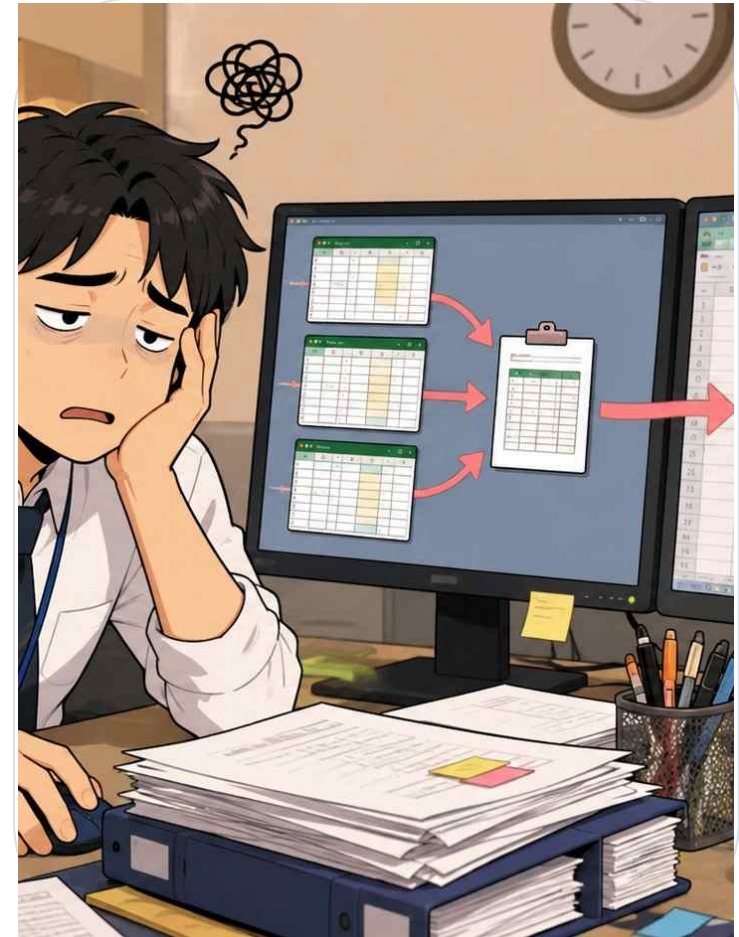
- 한글 문서에서 본문 텍스트 추출
- [표 숫자] 제목 찾기
- 목록 표시
- 사용자 선택
- 번호 수정 명령 생성
- 한글 문서에 반영



한글 자동화에서 자주 만나는 문제

오류가 생기면 단계별로 원인을 분리해야 합니다.

- 한글 프로그램이 열려 있지 않음
- 열려 있는 한글 객체를 못 찾음
- 문서가 저장되지 않은 상태
- 본문은 읽었지만 위치 이동 실패
- 같은 표 번호가 여러 개 있음
- 표 내부 텍스트와 본문 제목 구분 필요

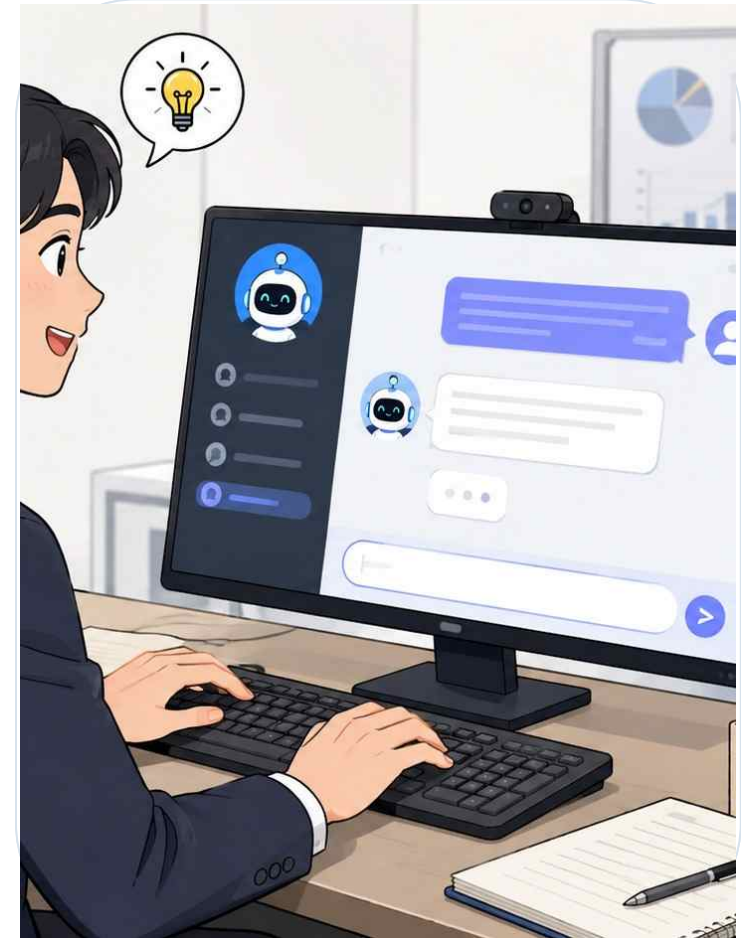


표도우미 요구사항 정리 프롬프트

Windows에서 Python으로 한글 문서의 표 번호를 관리하는 도구를 만들고 싶어.

최종 목표

1. 현재 열려 있는 한글 문서에서 본문 텍스트를 가져온다.
2. 본문에서 [표 숫자] 제목 형식의 문단을 찾는다.
3. 찾은 표 제목을 GUI 목록으로 보여준다.
4. 시작 번호 입력 후 n+ 또는 n-를 실행한다.
5. [표 0]은 새로 삽입한 표로 별도 처리한다.
6. 같은 번호가 여러 개 있을 수 있으므로 출현 순서와 위치를 관리한다.



오류를 알려주는 법

아래 오류가 발생했어.

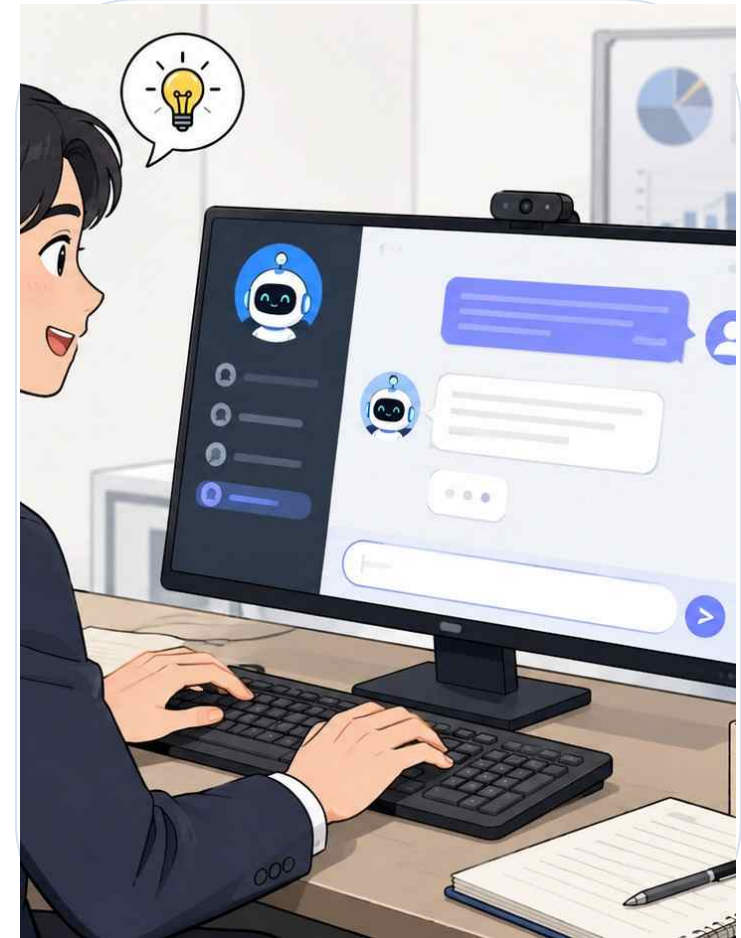
오류 메시지:

-2147023174 RPC 서버를 사용할 수 없습니다.

상황:

1. 한글 문서는 열려 있었어.
2. 프로그램 실행 후 한글 연결이 끊기는 것 같아.
3. 본문 읽기 전 단계에서 실패한 것 같아.

원인을 경우별로 나눠서 설명하고,
현재 열려 있는 한글 객체를 다시 찾는 방식으로 수정해줘.

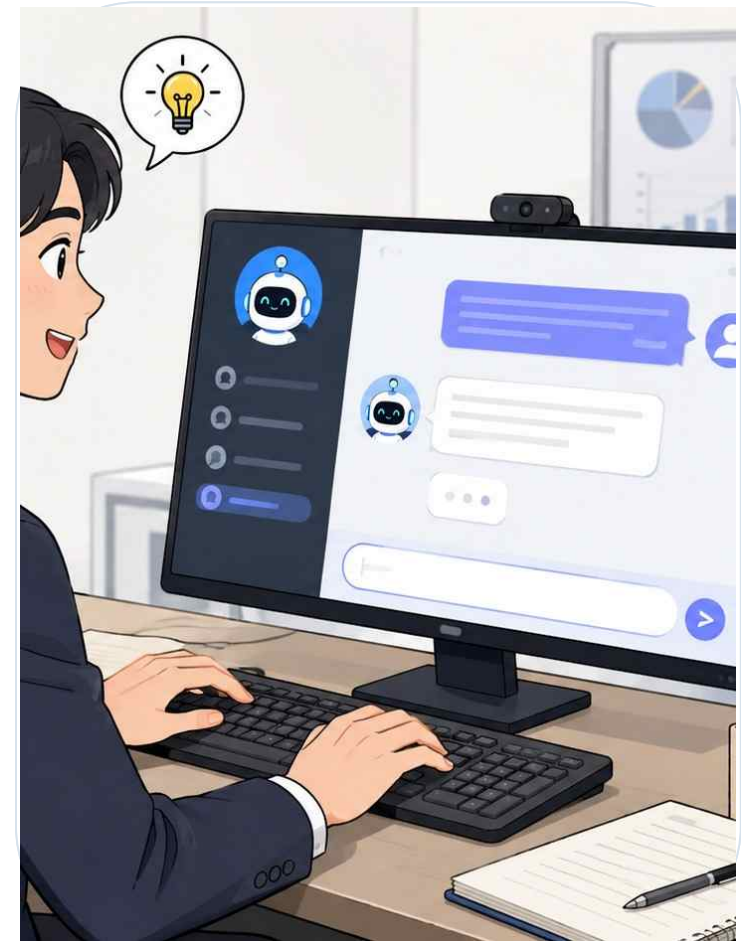


기능 개선 요청하기

현재 프로그램은 $n+$ 실행 후 프로그램이 종료되어 불편해.

원하는 동작

1. $n+$ 또는 $n-$ 실행 후에도 GUI 창은 닫히지 않게 해줘.
2. 실행 후 목록을 자동으로 갱신해줘.
3. 저장 버튼과 닫기 버튼은 아래쪽에 배치해줘.
4. 표 0 삽입 처리, $n+$, $n-$ 버튼은 위쪽에 배치해줘.



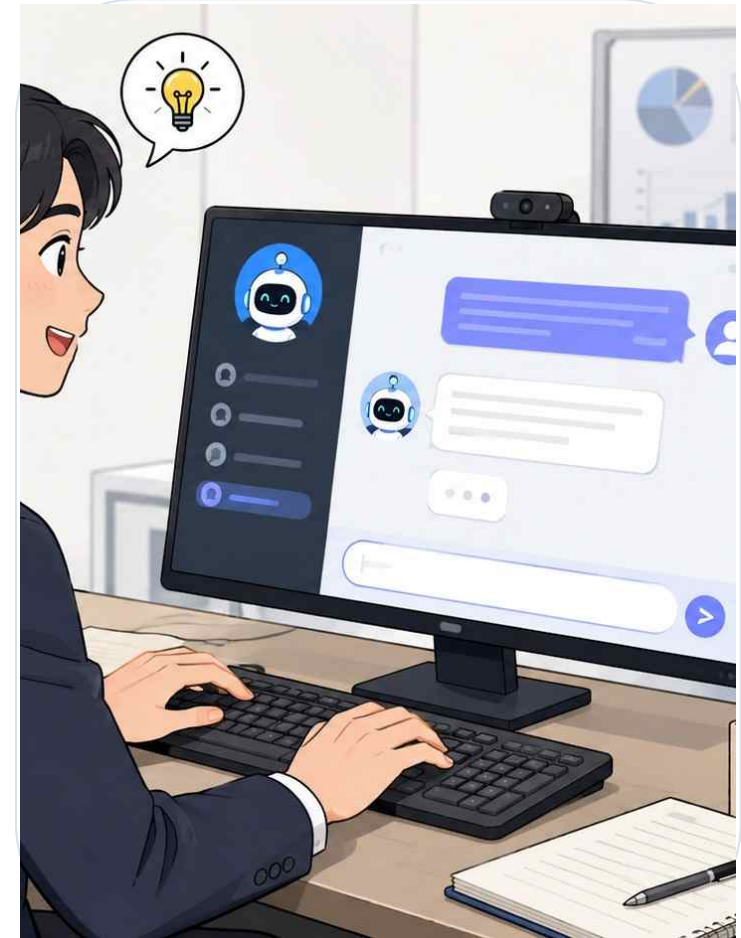
예외 처리 요청하기

같은 번호의 표가 여러 개 있을 수 있어.

예를 들어 본문에 [표 2]가 두 개 있을 때, 앞의 [표 2]는 그대로 두고 뒤의 [표 2]만 [표 3]으로 바꾸고 싶어.

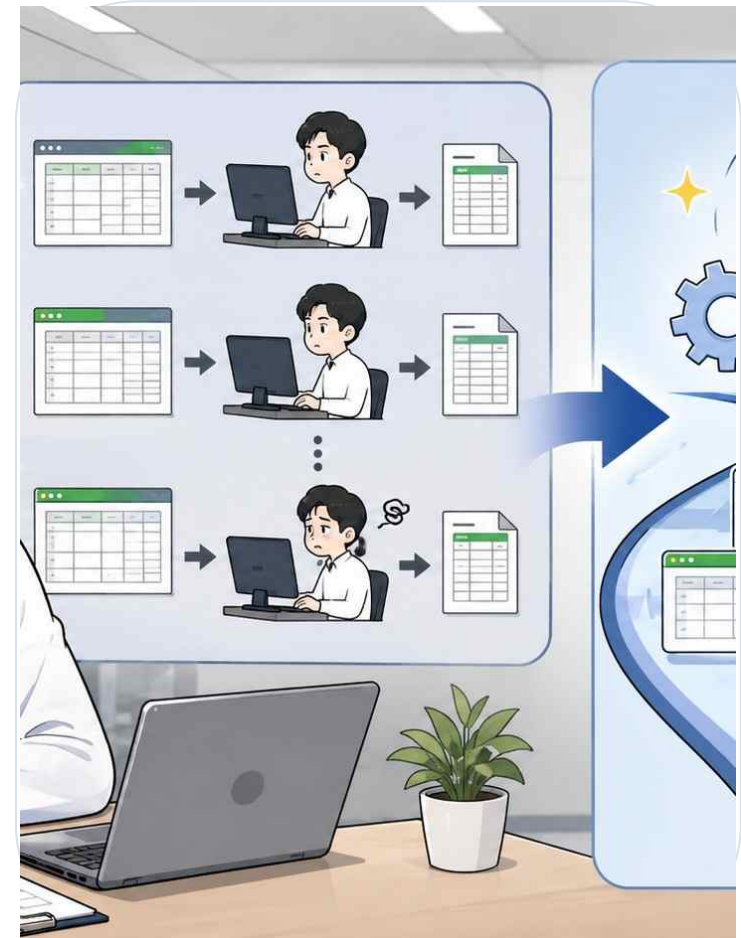
단순히 번호만 기준으로 바꾸면 앞의 표까지 같이 바뀌는 문제가 생겨.

표 번호뿐 아니라 문서에서 몇 번째로 출현했는지와 위치 정보를 함께 관리하도록 수정해줘.



표도우미 중급 과정의 핵심 구조

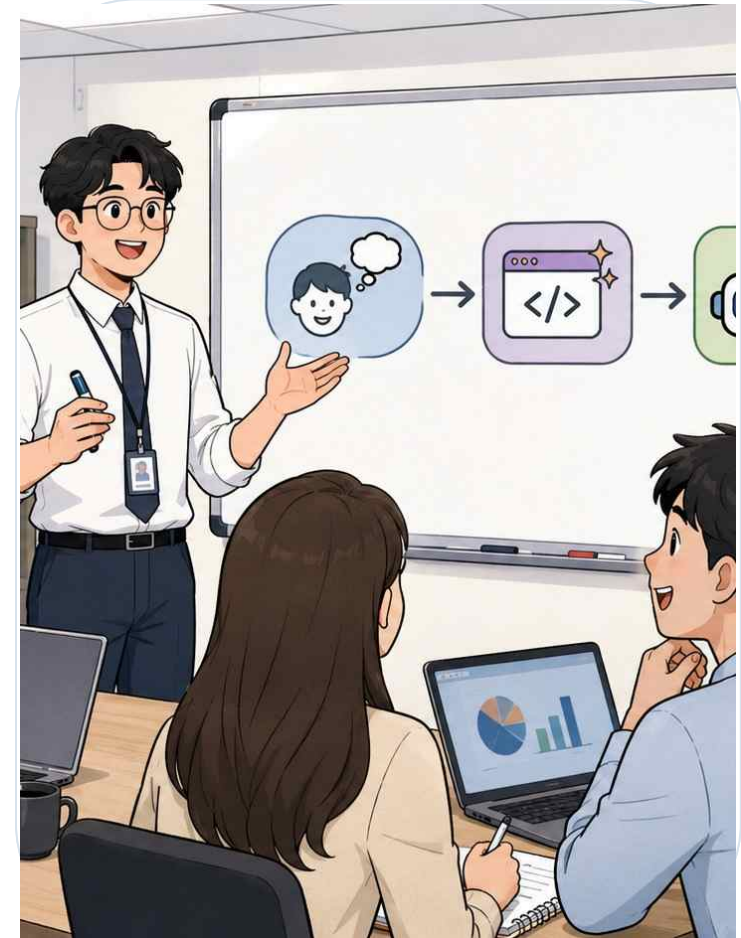
- 1 텍스트에서 [표 n] 찾기
- 2 표 번호 +1, -1 처리하기
- 3 GUI 만들기
- 4 한글 문서 텍스트 추출하기
- 5 한글 문서 위치 정보 관리하기
- 6 본문 수정 반영하기
- 7 오류 메시지 기반으로 개선하기
- 8 실제 업무 도구로 정리하기



중급 과정에서 강조할 점

한 번에 완성하려 하지 않는 것이 중요합니다.

- 먼저 텍스트 파일로 연습한다
- 그다음 GUI를 붙인다
- 마지막에 한글 문서 연결을 붙인다
- 오류 메시지를 그대로 기록한다
- 한 번에 여러 기능을 고치지 않는다
- 작동하는 버전을 보관해 둔다



전체 교육 마무리

구분	핵심 내용
초급	개발환경 세팅, 엑셀 취합, 오류 해결, 요구사항 변경
중급	표 번호 찾기, GUI, 한글 연결 구조, 예외 처리
공통	업무 절차를 설명하고 AI와 함께 고쳐 가기

바이브 코딩의 핵심은 코드를 외우는 것이 아니라, 반복 업무를 구조화해서 AI에게 설명하는 것입니다.



마지막 메시지

반복 작업은
컴퓨터에게,
검토와 판단은
사람에게.

개발환경 세팅을 넘으면
작은 자동화 도구를 하나씩 만들어 갈 수 있습니
다.

